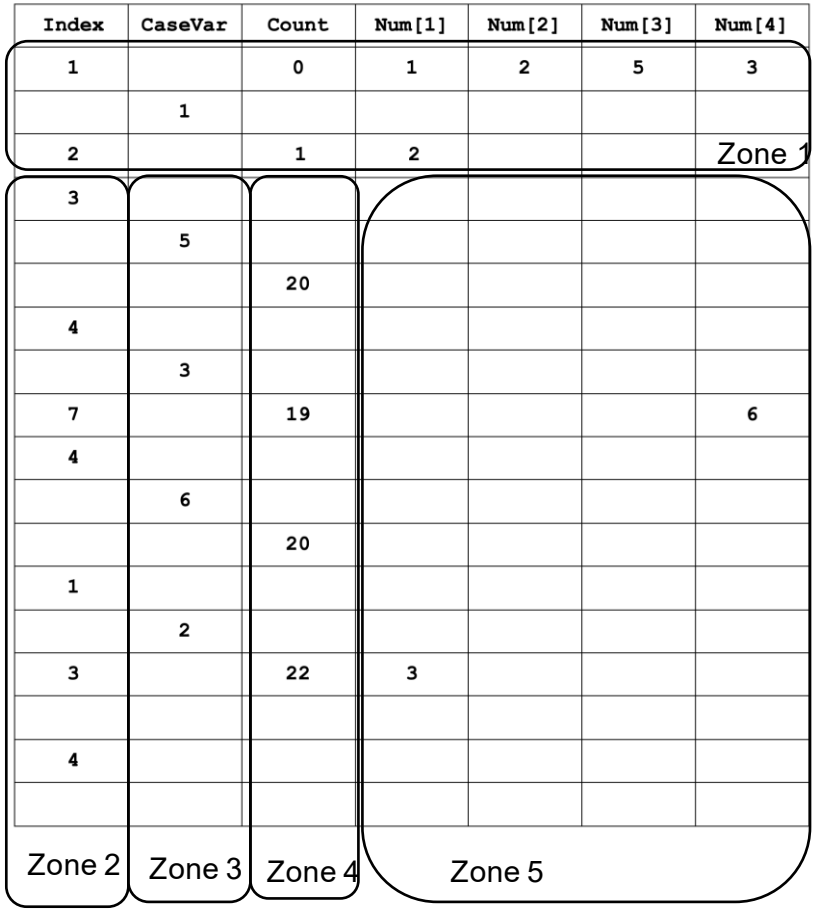
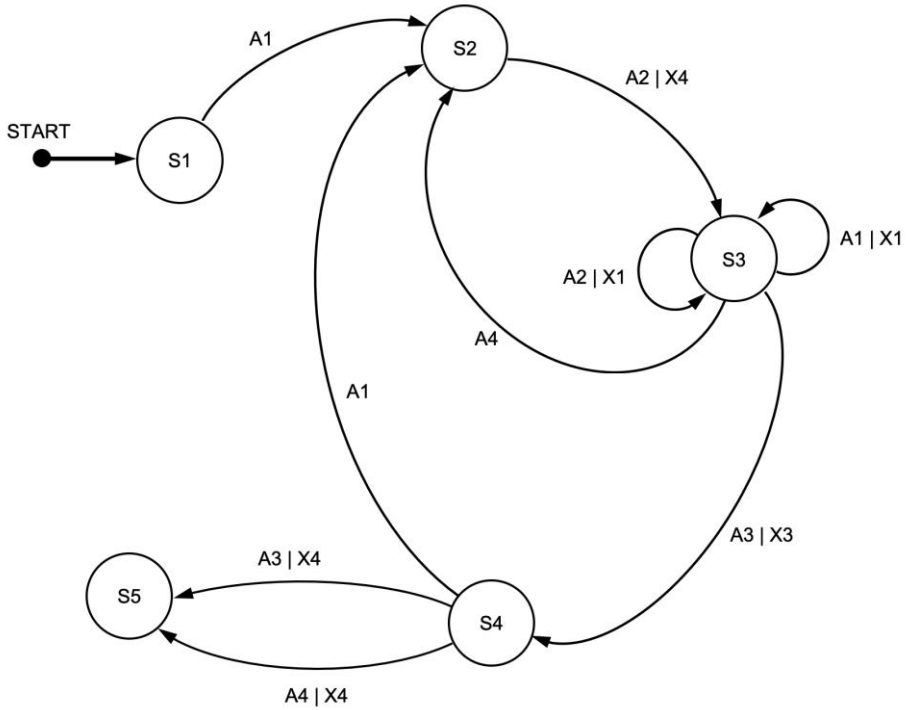
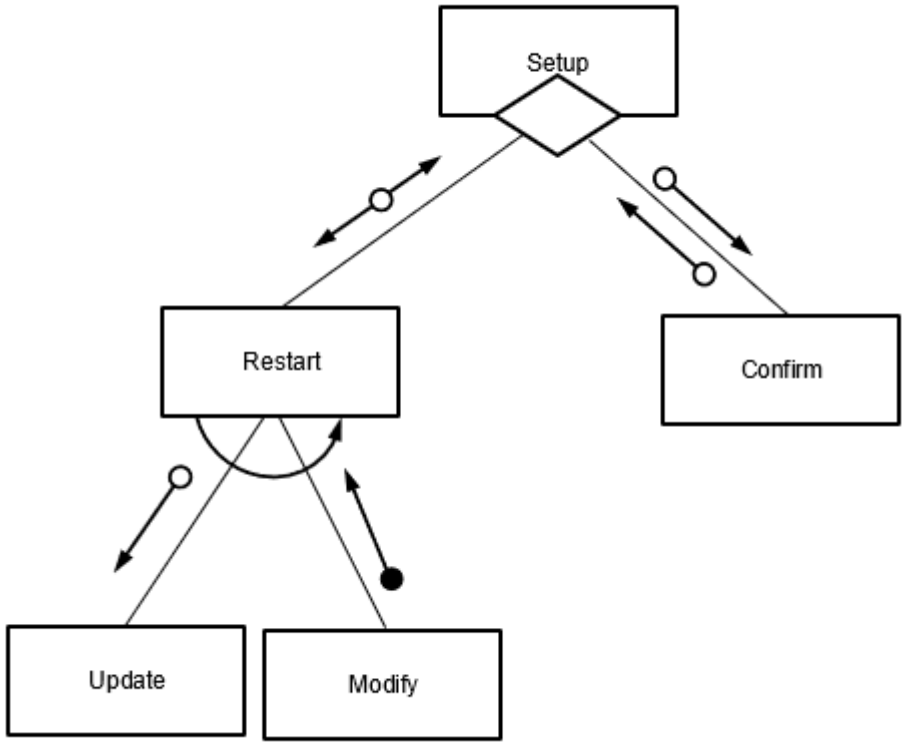
5(a)	 Zones 1 to 4 : One mark for each set of values as shown Zone 5: Num[4] set to 6 when Index is 7 **and** Num[1] set to 3 when index is 3 for **second** time	**5**

Examiner report missing
9618_w25_qp_23 Question 5

6(a)	One mark per point: 1 Create a list of numbers that should generate **all of the** possible check digit values 2 and check that each / the / a generated value is as expected	**2**

Examiner report missing
9618_w25_qp_23 Question 6

7(a)	 One mark for each: 1 S2 to S5 labelled 2 Label A1 added to event from S1 to S2 3 Event lines from S2 to S3 **and** S3 to S2 correctly labelled 4 Events lines from S3 to S3 **and** event line from S3 to S4 all labelled correctly 5 Event lines from S4 to S5 correctly **and** event line from S4 to S2 labelled correctly **Max 4** for any additional events/lines	**5**

7(b)	 One mark for each: 1 All **boxes** correctly labelled and connected in correct hierarchy 2 BYREF parameter to Restart, parameter to Confirm and return 3 Diamond and loop symbol 4 Parameter to Update **and** return from Modify	**4**

Examiner report missing
9618_w25_qp_23 Question 7

5(a)		**4**

Activity	Name of life cycle stage
a structure chart is produced	**Design**
a program is modified to allow it to run on new hardware	**Maintenance**
the programmer identifies the customer's requirements	**Analysis**
an Integrated Development Environment (IDE) provides context-sensitive help such as 'auto-complete'	**Coding**

One mark per row

5(b)	Acceptance testing	**1**

Examiner report missing
9618_w25_qp_22 Question 5

7(a)	Example explanation: The arrow means that `Convert` **repeatedly** calls `Analyse` then `Update` and then `Sort` (in that order)10 **MP1** Reference to 'repetition'…' **MP2** Naming **all <u>four</u> modules** correctly **and** the sequence is stated or implied	**2**

7(b)	**MP1** `PROCEDURE Analyse (Count : INTEGER)` `FUNCTION Update (Key : STRING) RETURNS INTEGER` **MP2** **MP3** **MP4** `PROCEDURE Sort (BYREF P2 : BOOLEAN)` Mark as follows: One mark per underlined part	**4**

Examiner report missing
9618_w25_qp_22 Question 7

8(a)	Example solution:
	
	`PROCEDURE CountLoans(ThisStudentID : STRING)`
	`  DECLARE Index, LCount, RCount : INTEGER`
	
	`  LCount ← 0`
	`  RCount ← 0`
	
	`  FOR Index ← 1 TO 7000`
	`  IF Loan[Index].StudentID = ThisStudentID THEN`
	`  IF Loan[Index].OnLoan = FALSE THEN`
	`  RCount ← RCount + 1`
	`  ELSE`
	`  LCount ← LCount + 1`
	`  ENDIF`
	`  ENDIF`
	`  NEXT Index`
	
	`  OUTPUT "Student has ", LCount, " books on loan"`
	`  OUTPUT "and has returned ", RCount, " books"`
	
	`ENDPROCEDURE`
	
	Mark as follows:
	MP1 Declaration of Index **and** two count variables
	MP2 Loop for 7000 iterations
	MP3 `Index` references an **indexed data** item with correct **.dot notation**
	MP4 Attempt to compare the `StudentID` field = **parameter in a loop**
	MP5 Test of the `OnLoan` field **and** correct logic for both outcomes
	MP6 Both count variables initialised **and** increment appropriate count **in a loop**
	MP7 Output includes both counts with a <u>suitable</u> message
		7

8(d)	Additional date(s) solution **MP1** Store the **return date** of each loan // the **start date <u>and</u>** the **loan period** **MP2** Algorithm will do a **calculation** involving the MP1 data item(s) OR The student email address solution **MP3** Store the student <u>email address</u> **MP4** In order **to send** the email // the student is alerted to return the book **Max 2**	**2**

Examiner report missing
9618_w25_qp_22 Question 8

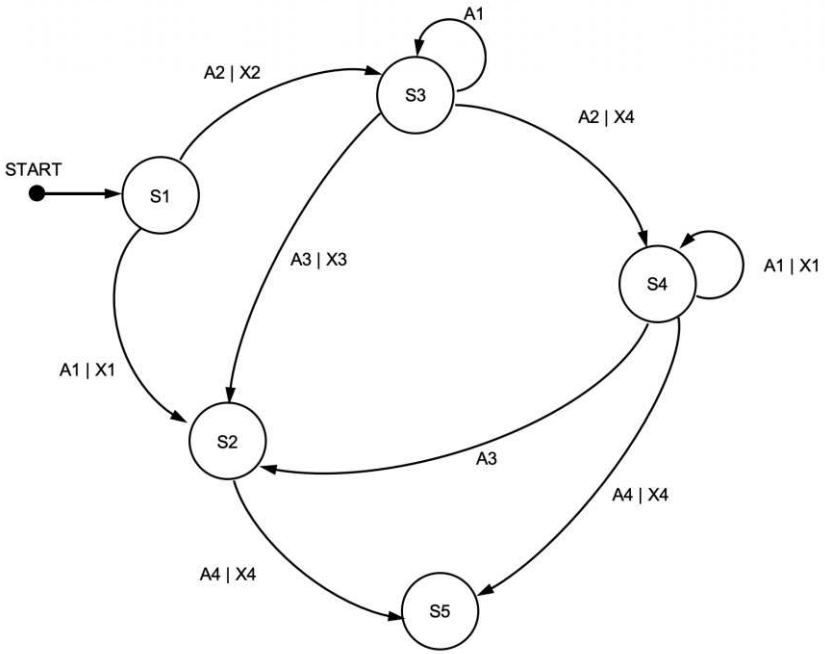
1(a)	<table><tr><th>Expression</th></tr><tr><td>MyString ← 'X' & MyString</td></tr><tr><td>MyChar ← MID ("ABCD", 1 / 2 / 3 / 4 , 1)</td></tr><tr><td>MyString ← NUM_TO_STR (DAY / MONTH / YEAR / DAYINDEX (MyDOB))</td></tr><tr><td>MyInt ← INT (LENGTH (MyString) / 2)</td></tr></table> One mark per row	**4**
1(b)	<table><tr><th>Test description</th><th>Test method</th></tr><tr><td>carried out as soon as a program module has been coded</td><td>alpha</td></tr><tr><td>carried out as program modules are being combined</td><td>integration</td></tr><tr><td>completed by the developers without referring to the code</td><td>black-box</td></tr><tr><td>completed by the customer</td><td>acceptance / beta</td></tr></table> One mark per row (2 to 4)	**3**

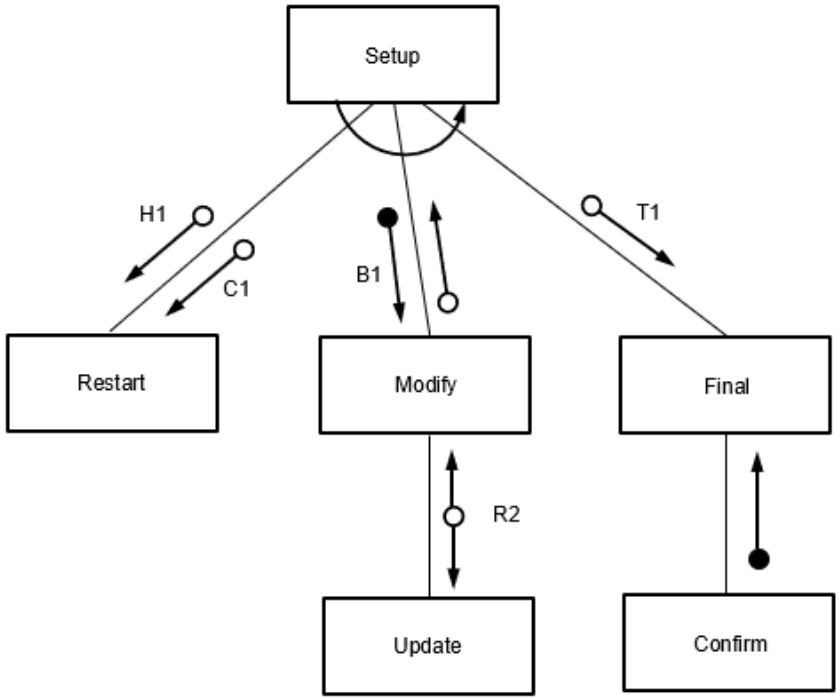
1(c)	One mark per point: 1 reference to a **breakpoint** 2 reference to **single stepping** 3 Correct explanation of **both** e.g. to stop program execution at specific point / line / statement / instruction and to execute one line / statement / instruction at a time to (locate an/the error) One mark for each bulleted point	**3**

Examiner report missing
9618_w25_qp_21 Question 1

6(a)	Number:	Any number where the first two digits sum to 5 / 15 **and** the last three digits are 623	**2**
	Explanation:	The remainder is the same / 3 // The sum of the (first four) digits is the same / 13	

Examiner report missing
9618_w25_qp_21 Question 6

7(a)	 One mark for each: 1 Label A1 | X1 added on line from S1 to state labelled S2 2 States S3, S4 and S5 labelled 3 Line from S2 to S5 with event label A4|X4 4 Lines from S3 with correct event labels including 'loop' label 5 Lines from S4 with correct event labels including 'loop'	**5**

7(b)	 One mark per point: 1 All boxes correctly labelled 2 Iteration arrow 3 Parameter to Restart 4 Return value from Modify **and** parameter to Final 5 BYREF parameter to Update	**5**

Examiner report missing
9618_w25_qp_21 Question 7

8(a)	Example solution:
	
	`PROCEDURE OKToBorrow(ThisStudentID : STRING)`
	`  DECLARE Index, Count : INTEGER`
	
	`  CONSTANT Max = 5`
	`  Count ← 0`
	`  Index ← 1 / 0`
	
	`  WHILE Index <= 5000 / Index <= 4999 AND Count < Max`
	`  IF Loan[Index].StudentID = ThisStudentID AND __`
	`  Loan[Index].OnLoan =`
	`TRUE THEN`
	`  Count ← Count + 1`
	`  ENDIF`
	`  Index ← Index + 1`
	`  ENDWHILE`
	
	`  IF Count < Max THEN`
	`  OUTPUT "Student may borrow another book"`
	`  ELSE`
	`  OUTPUT "Student may not borrow another book"`
	`  ENDIF`
	
	`ENDPROCEDURE`
	
	Mark as follows:
	1 Procedure heading, parameter and ending
	2 Loop through **all elements** in `Loan` array
	3 ... terminating if `Max` reached
	
	4 Test for correct `StudentID` field **in a loop**
	5 ... and that `OnLoan = TRUE` **in a loop**
	... if so, then increment `Count` **in a loop**
	6 Output an appropriate **message** in both cases **after the loop**
		7

8(c)	Award marks for reference to following design features: Change to existing Record: 1 Store date due back as a new data item / in the loan record 2 Store date of loan **and** the loan length category as new data items / in the loan record **Max 1** New Record 3 **New** Records with Category (and Loan length field) Change to Program: 4 (Use the loan length category of a book to) calculate the date due date 5 Compare due date with today's date (to check if overdue / to calculate fine) 6 Map loan length category to length of loan / date due back **Max 1** **Max 2 marks**	**2**

Examiner report missing
9618_w25_qp_21 Question 8

4(a)	**MP1** **Open** the file (`Sales.txt`) in **read mode and** subsequently close **MP2** **Read <u>a line</u>** (from the text file) **MP3** Convert the line read (string) to a number **MP4** If the number is greater than 500 **and** then output week number **MP5** Increment week number (must have been Initialised **…**) **MP6** Repeat from step 2 for 52 times // Repeat from step 2 until end of file **Max 5**	**5**

4(b)	**MP1** Going through the program **a line / statement at a time** **//** doing a **<u>dry run</u>** // **<u>single-stepping</u>** **(**at run-time with an IDE) // Peer testing/review is carried out **MP2** Creating a **<u>trace table</u>** **MP3** Draw up **test data** // draw up list **of inputs with expected output** **//** boundary, normal, abnormal data is used **MP4** Errors will be indicated when a **variable** / **output** gives an **unexpected value** **MP5** Faults in the logic of the program can be detected // Errors may be indicated by an unexpected path through the program **Max 3**	**3**

Examiner report missing
9618_s25_qp_23 Question 4

6(a)	**MP1** A function returns a (single) **value and** a procedure does not **MP2** A procedure can pass parameters by ByRef **Max 1**	**1**

6(b)	Sub_Part1A:
	
	`PROCEDURE Sub_Part1A(R: INTEGER)` 1 Mark
	
	Sub_Part2A:
	
	`FUNCTION Sub_Part2A(T : REAL) RETURNS BOOLEAN`
	
	2 Marks
	
	Sub_Part3A:
	
	`PROCEDURE Sub_Part3A(V: INTEGER, W: BOOLEAN, BYREF U : STRING)`
	
	2 Marks
		5

6(c)		**3**

Symbol	Explanation
	The module **Main** calls either **Sub_A or Sub_B** (which one is called is determined at run time)
	The module **Sub_A** will repeatedly call **Sub_Part1A** followed by **Sub_Part2A** then **by Sub_Part3A**

MP1 Shape1: reference to **selection**
MP2 Shape2: reference to **repetition // iteration**

MP3 **Descption (×2)** including correct use of **all module names** in **both** parts

Examiner report missing
9618_s25_qp_23 Question 6

7(a)(i)	Example solution:
	
	`PROCEDURE UpdateVisit(BYREF LastVisitDate : DATE,__`
	`BYREF LoyaltyPoints : INTEGER)`
	
	`IF MONTH(TODAY()) = MONTH(LastVisitDate) THEN`
	`LoyaltyPoints ← LoyaltyPoints + 4`
	`ELSE`
	`LoyaltyPoints ← LoyaltyPoints + 1`
	`ENDIF`
	
	`LastVisitDate ← TODAY()`
	
	`ENDPROCEDURE`
	
	MP1 Procedure heading **and** ending **and** two parameters of correct type
	MP2 … both passed `BYREF`
	
	MP3 Use of `TODAY()` function to obtain current date
	MP4 Use `MONTH` function (×2) **and** one parameter is the header parameter **and** 2nd parameter is `TODAY()` / assigned variable
	
	MP5 If visit is same month, increase `LoyaltyPoints` by 4 otherwise increase by 1
	MP6 `LastVisitDate` set to `TODAY()` / 'current date' variable (assigned or unassigned)
	Max 5
		5

7(a)(ii)	`DAYINDEX(LastVisitDate) = DAYINDEX(TODAY())`	**1**

Examiner report missing
9618_s25_qp_23 Question 7

1(a)(i)	One mark for Choice of program development cycle depends on the **approach/method** required to produce the program **Or** One mark for a factor that would influence the choice of program development cycle, e.g. • Rapid development of program/prototypes required • Need for prototypes (at an early stage) • Complexity of problem • Skills / experience of development team / programmer(s) • Budget / Time / Resources available • Size of development team • Developer / programmer can return to earlier **stages** / any stage • Avoidance of repeating previous stages in development of program • How much involvement clients will have • Allows the (program) requirements to be changed during program development • (Program) requirements agreed at start of development of program **Max 1**	**1**

1(a)(ii)	Which programming language would best be suited to control the production line // Which programming language would best suit the problem being solved	**1**

1(b)(i)	Examples include: • Change to (production line) requirements • New technology / hardware available (to control production line) • Changes made to library modules **used** • Change in relevant legislation **Max 3**	**3**

1(b)(ii)	Perfective // Corrective	**1**

Examiner report missing
9618_s25_qp_22 Question 1

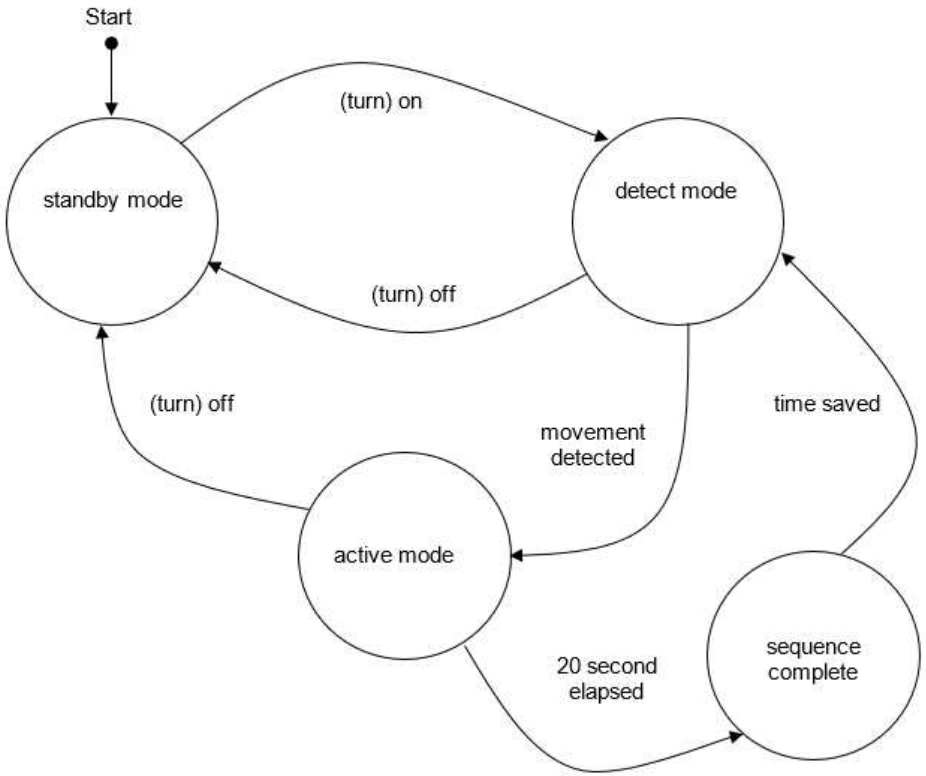
2(a)	Mark as follows: 1. To increase the level of detail of the algorithm // To break the problem / task into **smaller steps** … 2. … until steps can be directly translated into lines of code // from which it can be programmed	**2**

2(b)(i)	One mark for each of set test values Hours worked between 1 and 40 inclusive Sales value <= 2000 Bonus Pay: 0 Hours worked above 40 Sales value <= 2000 Bonus Pay: 10 Hours worked between 1 and 40 inclusive Sales value > 2000 Bonus Pay: 50 Hours worked above 40 Sales value above 2000 Bonus Pay: 100 **max 2**	**2**

2(b)(ii)	One mark per point • Simple **modules** are written to **replace** each of the unfinished modules. • Each simple module will return an expected value / will output a message to show it has been called.	**2**

2(b)(iii)	One mark for naming type of error and one mark for corresponding description Type of error: Logic (error) Description: Where the program does not behave as expected / Does not give expected result / An error in the logic of the algorithm **Or** Type of error: Run-time (error) Description: The program performs an illegal operation **Max 2**	**2**

Examiner report missing
9618_s25_qp_22 Question 2

5(a)	 Mark as follows: 1 Mark: The three new states drawn and labelled. 1 Mark: Two of the events drawn with labels connecting correct states and correct line direction 1 Mark: Four of the events drawn with labels connecting correct states and correct line direction. 1 Mark: All and only six events drawn with labels connecting correct states and correct line direction.	**4**

Examiner report missing
9618_s25_qp_22 Question 5

6(a)(i)		**4**

Value	Start	Unused	New	Last	Current	
1043	2	8	8	−1	2	Region 1
				2	3	Region 2
				3	1	Region 3
				1	7	
				7	4	Region 4

Award 1 mark per region

6(a)(ii)		**3**

	Data
1	1018
2	1007
3	1010
4	1056
5	1092
6	1062
7	1034
8	**1043**
9	0
10	0

	Pointer
1	**7**
2	**3**
3	**1**
4	**6**
5	**–1**
6	**5**
7	**8**
8	**4**
9	**10**
10	**–1**

Mark as follows:

1 mark for global array `Data` row 8 containing the value 1043

1 mark for global array `Pointer` row 7 containing the value 8 and row 8 containing the value 4

1 mark for all other rows in both arrays

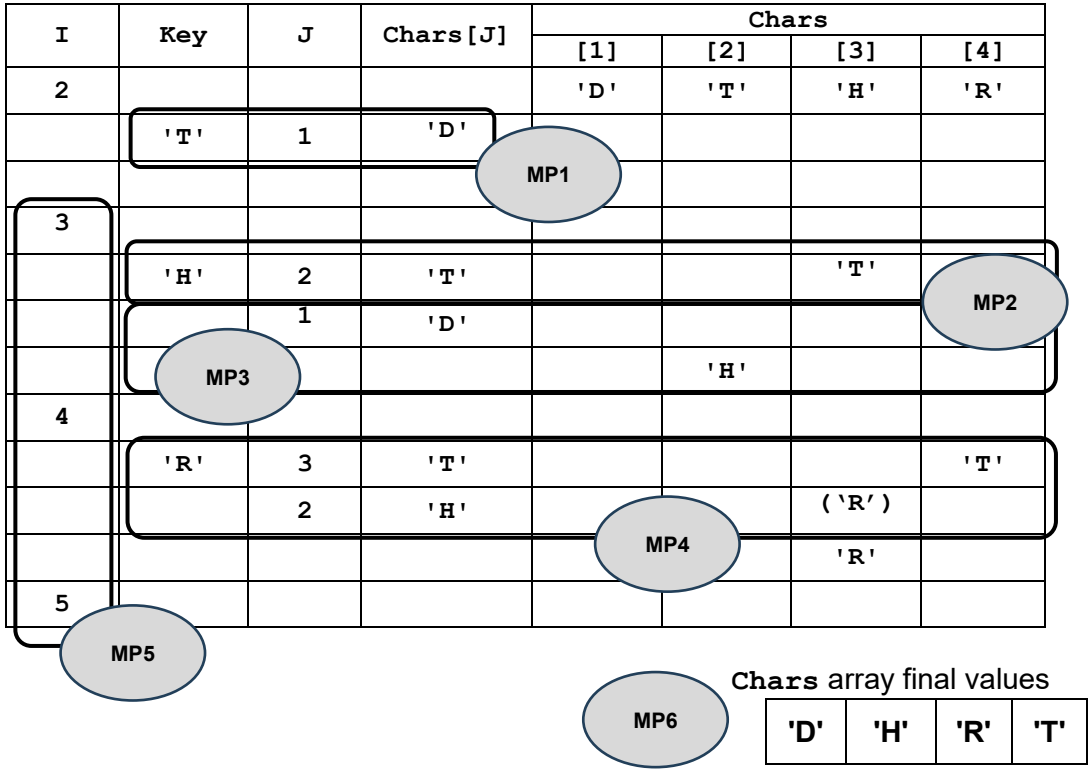
Examiner report missing
9618_s25_qp_22 Question 6

2(a)(i)	**MP1** The two decision boxes are in the wrong order // The **first decision** symbol is wrong **MP2** The first decision should check for values greater than or equal to 2000 **Max 1**	**1**
2(a)(ii)	Explaining how to correct the flowchart. **MP1** Swap around the two decision boxes **MP2** The first decision box should check for values greater than or equal to 2000 **and** the second decision box should check for values above 4000 **Max 1**	**1**
2(b)(i)	2000 / 4000 / 10 / 100	**1**
2(b)(ii)	**MP1** The value **cannot be changed** // avoids being different in two places **MP2** Makes the program easier to understand **MP3** Avoids repeatedly writing the same value throughout the program **MP4** A change to the value requires a change in only one statement **Max 2**	**2**
2(b)(iii)	**MP1** (The code is) tried and tested so error free **MP2** Provides functionality / code that the programmer may find difficult to code themselves **Max 1**	**1**

2(c)	**MP1** **Syntax (error)** **MP2** Rules of programming language // the language grammar was not followed **MP3** **Run-time (error**) **MP4** The program performs an illegal operation Mark as follows: One mark for naming each type of error **and** one mark for the description	**4**

Examiner report missing
9618_s25_qp_21 Question 2

4(a)	**MP1** Count-controlled **MP2** The number of iterations required is known	**2**

4(b)	 Mark as follows: **MP1, MP2, MP3, MP4, MP5** 1 mark per enclosure	**6**

Examiner report missing
9618_s25_qp_21 Question 4

6(a)		**4**

Type of test data	Test data value	Expected outcome
normal	36	data item is accepted
boundary/ extreme/ normal	0 / 1	data item is accepted
boundary/ extreme/ normal	59 / 60	data item is accepted
abnormal	>= 61	data item is rejected
normal	15	data item is accepted

Mark as follows:
1 mark for each row with (Type **and** Value **and** Outcome)

Examiner report missing
9618_s25_qp_21 Question 6

7(a)	Example solution:
	
	`FUNCTION CustomerOrder(Number, Points : INTEGER) RETURNS INTEGER`
	
	`   DECLARE NewPoints, NumberFree: INTEGER`
	
	`   NewPoints ← Points + Number`
	`   NumberFree ← 0`
	`   WHILE NewPoints >= 11 AND Number > 0`
	`      NumberFree ← NumberFree + 1`
	`      NewPoints ← NewPoints -11`
	`       Number ← Number - 1`
	`   END WHILE`
	
	`   OUTPUT "Number of free coffees is: ", NumberFree`
	`   RETURN NewPoints`
	`ENDFUNCTION`
	
	MP1 Function header **and** ending **and** parameters **and** return type
	MP2 Number of coffees ordered added to points
	
	MP3 Correct calculation of one free coffee
	
	MP4 Attempted calculation of multiple free coffees **and** reduced `NewPoints`
	
	MP5 Output number of free drinks with suitable message
	
	MP6 Return of `NewPoints`
		6

7(b)(i)	Example solution:
	
	`PROCEDURE AddNewCustomers(NumToAdd : INTEGER)`
	
	`   DECLARE CustomerID : INTEGER`
	`   DECLARE Count : INTEGER`
	`   DECLARE Line : STRING`
	`   DECLARE NewLine : STRING`
	`   OPENFILE "Loyalty.txt" FOR READ`
	
	`   WHILE NOT EOF("Loyalty.txt")`
	`      READFILE "Loyalty.txt", Line`
	`   ENDWHILE`
	`   CLOSEFILE "Loyalty.txt"`
	
	`   OPENFILE "Loyalty.txt" FOR APPEND`
	
	`   CustomerID ← STR_TO_NUM((LEFT(Line, 6))`
	
	`   FOR Count ← 1 TO NumToAdd`
	`      CustomerID ← CustomerID + 1`
	`      OUTPUT CustomerID`
	`      NewLine ← NUM_TO_STR(CustomerID) & ",0"`
	`      WRITEFILE "Loyalty.txt", NewLine`
	`   NEXT Count`
	
	`   CLOSEFILE "Loyalty.txt"`
	`ENDPROCEDURE`
	
	Mark as follows:
	MP1 All variables used are declared using the correct type including a string **and** an integer
	MP2 Open file in read mode **and** close (before opening in append mode)
	
	MP3 Conditional loop with `EOF("Loyalty.txt")`
	MP4 Read `Line` from file // Count number of records in the file
	
	MP5 Open the file in append mode **and** subsequently close
	MP6 Extract `CustomerID` from `Line` **and** convert to integer // use count from MP4 to generate last `CustomerID` stored
	MP7 A (count controlled) loop for the number of customers to add
	MP8 Increment `CustomerID` **and** output the new `CustomerID` in loop
	
	MP9 Create string for new `CustomerID` **and** write to file in loop
	
	Max 8
		8

7(b)(ii)	**MP1** **MP2** **MP3** **MP4** **MP5** **Max 4**	Check if the text file `loyalty.txt` exists / is empty If file does not exist it must be created (using write mode) If the file has to be created / is empty then set the first `CustomerID` to `100001` Write `"100001,0"` to `loyalty.txt` (using write mode) For all but the first customer (to be added to the empty file) **use the existing code / module**	**4**

Examiner report missing
9618_s25_qp_21 Question 7

5(a)	<table><tr><th>Activity</th><th>Name of life cycle stage</th></tr><tr><td>A compiler is be used.</td><td>Coding</td></tr><tr><td>A program that has been released for general use is modified</td><td>Maintenance</td></tr><tr><td>The dry run method is used.</td><td>Testing</td></tr><tr><td>The program structure is specified.</td><td>Design</td></tr></table> One mark per row	**4**

5(b)	One mark for method – two marks for the description **MP1** Stub testing **MP2** The modules `Test_A()` and `Test_B()` are replaced by dummy modules **MP3** ... which return a known result // An output statement is displayed when called (to check it works) // gives expected output	**3**

Examiner report missing
9618_w24_qp_23 Question 5

7(a)	One mark per answer: **MP1** Analysis **MP2** **Named** document, examples include: • A requirement specification • Definition of System **Objectives** • List of problems with existing system • Survey results • Feasibility study • Interview notes **Max 2 marks**	**2**

Examiner report missing
9618_w24_qp_23 Question 7

1(a)	(Corrective) Maintenance	**1**

1(b)	For example: • Set a breakpoint at the start of / within `Lookup,` to stop execution at a given statement • then use single stepping to execute one statement / instruction at a time • to display the value of variables using a watch window One mark for each: **MP1** Order starting with a breakpoint **and** an explanation – 'stop execution at this statement / line' **MP2** Explanation of single stepping – execute 'line by line' / statements **MP3** Explanation of watch window – displaying the value of variable(s)	**3**

1(c)	Features include: **MP1** Editor **MP2** Auto- (syntax) complete / auto correction // identify undeclared variables(s) **MP3** Prettyprint / auto-indentation / auto (structure) highlighter **MP4** Dynamic syntax checking **MP5** Expand / collapse code blocks **MP6** Context sensitive prompts **Max 2 marks**	**2**

Examiner report missing
9618_w24_qp_22 Question 1

5(a)	**MP1** `Count ← INT(100 / Number)` Number could be zero (giving a divide by zero) **MP2** `Index ← Data[Number]` Potential error: Value `Number` could be outside the range of array indices **MP3** `ReturnValue ← TO_UPPER(RIGHT(Label, Count))` Potential Error: Number to extract may be too big / negative / out of range for use in the `RIGHT` function // `Label` has insufficient characters **MP4** `RETURN RetVal` Potential Error: There is no value to be returned // there is no variable named `RetVal` Mark as follows: 1 mark for each statement **and** description **Max 3 marks**	**3**

5(b)	**MP1** Construct: A (pre/post) conditional loop **MP2** Explanation: The terminating condition is never satisfied	**2**

Examiner report missing
9618_w24_qp_22 Question 5

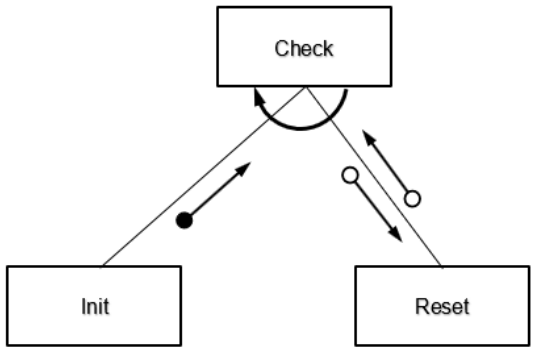
6(a)	Example solution:
	
	`PROCEDURE Special()`
	`  DECLARE Index : INTEGER`
	`  DECLARE Filling1, Filling2 : STRING`
	
	`  REPEAT`
	`  Index ← INT(RAND(35)) + 1`
	`  UNTIL Filling[Index] <> ""`
	
	`  Filling1 ← Filling[Index]`
	
	`  REPEAT`
	`  Index ← INT(RAND(35)) + 1`
	`  UNTIL Filling[Index] <> "" AND Filling1 <> Filling[Index]`
	
	`  Filling2 ← Filling[Index]`
	
	`  REPEAT`
	`  Index ← INT(RAND(10) + 1)`
	`  UNTIL Bread[Index] <> ""`
	
	`  OUTPUT "The daily special is ", Filling1, " and ", __ Filling2, " on ", Bread[Index], " bread."`
	
	`ENDPROCEDURE`
	
	Mark as follows:
	MP1 Loop for Filling 1, avoiding unused elements
	MP2 Loop for Filling 2 avoiding unused elements
	MP3 Check Filling 2 is different from Filling 1 – could correctly compare either the indices or the array contents
	
	MP4 Loop for Bread, avoiding unused elements
	
	MP5 Using `RAND(10) / RAND(35)`
	MP6 Completely correct use of `RAND()` – including `INT()` and +1 in all cases
	
	MP7 Correct output - **once only** – following a reasonable attempt at selection of filings and bread
		7

6(b)	Answers include: **MP1** For each filling, create a list of acceptable / incompatible fillings/indexes **MP2** When selecting the second filling, (as well as checking for an unused element) check that the filling / index is / is not on the list ALTERNATIVE: **MP1** Create a list of 'good' combinations **MP2** Randomly select from this list	**2**

Examiner report missing
9618_w24_qp_22 Question 6

7(a)	• Customer ID – to reference the other customer details • Email address – to send the email • Name – to personalise the email • Date of last visit – to select which customers should receive an email • Unique voucher code (or method of code generation) – to include in the email Mark as follows: One mark per item **and** justification **Max 3 marks**	**3**

7(c)	**MP1** Data structures // data dictionary // identifier table(s) // validation rules **MP2** Data-flow diagram // state-transition diagram **MP3** User interface // Format for the email **MP4** Testing method / Test plan / Test data / Trace tables **MP5** Choice of email protocol to be used // Programming language to be used // Development environment **MP6** Use of library routines // program to send the email **Max 3 marks**	**3**

7(d)	 **MP1** Three boxes correctly labelled **and** correct hierarchy **MP2** Parameter **and** return values **MP3** Iteration arrow	**3**

Examiner report missing
9618_w24_qp_22 Question 7

8(a)	Example solution:
	
	`PROCEDURE Assign(ThisRole : STRING, ThisPlayer : INTEGER)`
	`  DECLARE Index : INTEGER`
	`  DECLARE Done : BOOLEAN`
	
	`  Done ← FALSE`
	`  Index ← 1`
	
	`  WHILE Index < 46 AND Done = FALSE`
	`  IF Character[Index].Player = 0 AND __`
	`  Character[Index].Role = ThisRole THEN`
	`  Character[Index].Player ← ThisPlayer`
	`  Done ← TRUE`
	`  ELSE`
	`  Index ← Index + 1`
	`  ENDIF`
	`  ENDWHILE`
	
	`  IF Done = TRUE THEN`
	`  OUTPUT Character[Index].Name, " the ",__`
	`  Character[Index].Role, __`
	`  " has been assigned to player ", ThisPlayer`
	`  ELSE`
	`  OUTPUT "No characters with this role are available"`
	`  ENDIF`
	
	`ENDPROCEDURE`
	
	Mark as follows:
	MP1 Loop until 'found' or all 45 elements considered
	MP2 Test of `Player` field – i.e. not value **in a loop**
	
	MP3 ... `AND Role` **–** i.e. match for **`ThisRole`** parameter **in a loop**
	
	MP4 If available character found, assign `ThisPlayer` to the character **in a loop**
	MP5 When character found set termination condition/flag
	MP6 **Both** OUTPUT messages logically correctly placed
	
	MP7 **Both** OUTPUT statements correctly formed
		7

8(b)	Example solution:
	
	`PROCEDURE Save()`
	`  DECLARE Index  : INTEGER`
	`  DECLARE Line  : STRING`
	`  CONSTANT SEP = '^'`
	
	`  OPENFILE "SaveFile.txt" FOR WRITE`
	`  FOR Index ← 1 TO 45`
	`  Line ← NUM_TO_STR(Character[Index].Player) & SEP`
	`  Line ← Line & Character[Index].Role & SEP`
	`  Line ← Line & Character[Index].Name & SEP`
	`  Line ← Line & NUM_TO_STR(Character[Index].Level)`
	`  WRITEFILE "SaveFile.txt", Line`
	`  NEXT Index`
	
	`  CLOSEFILE "SaveFile.txt"`
	`ENDPROCEDURE`
	
	Mark as follows:
	MP1 Declaration of local integer for `Index` (**and** string type for `Line`)
	MP2 Open `"SaveFile.txt"` in write mode **and** subsequently close
	MP3 Loop through 45 elements
	
	MP4 Attempt to form `Line` - four fields **in a loop**
	MP5 Correct use of `NUM_TO_STR()x2 (Player and Level)` **in a loop**
	MP6 Correct use of **three** &<separator>& `strings` **in a loop**
	
	MP7 Line from MP4 written to file **in a loop**
		7

8(d)	**MP1** Method: Create a filename **suffix** which is **incremented** for each file save **MP2** Example: SaveFile01.txt, SaveFile02.txt	**2**

Examiner report missing
9618_w24_qp_22 Question 8

1(a)	One mark per point: Error: The **calculation** of TaxPayable is incorrect Correction: TaxPayable ← (ItemCost * TaxRate) / 100	**2**

Examiner report missing
9618_w24_qp_21 Question 1

3(a)	One mark per emboldened part: 1 Open the file **(Result.txt) in <u>read</u> mode** 2 Loop until **EOF(Result.txt)** // **EOF** // **end of file** 3 **<u>Read a</u> / <u>the</u> / <u>next line</u> (from the file)** and store in `ThisLine` 4 Assign `LineY` to `LineX` 5 Assign `LineZ` to `LineY` 6 Assign `ThisLine` to `LineZ` 7 After the loop, **close the file (Result.txt)** 8 Output `LineX`, `LineY`, `LineZ`	**4**

3(c)	Two loop solution One mark per point: 1 Loop until given line number (-1) / `<parameter> (-1)` (lines have been read) 2 ... if end of file is reached then return FALSE 3 Loop for three lines // Read three lines // Repeat of 4 and 5 for three lines 4 Read a line and output it 5 ... if end of file is reached then return FALSE 6 After outputting the (required) lines return TRUE 7 Ordering of lines no longer needed **Max 4 marks** Alternative solution **One loop solution mark scheme that reads to given line number + 2** One mark per point 1 Loop until given line number + 2 / `<parameter> + 2` (lines have been read) 2 **OR** end of file is reached 3 Return FALSE if EOF reached 4 Read a line from the file **in the loop** 5 Continue to order the last three lines read i**n the loop** // Steps 4 to 6 stay the same 6 (If FALSE has **not** been returned) output the required three lines (in the correct order) 7 … **and** Return TRUE **Max 4 marks**	**4**

Examiner report missing
9618_w24_qp_21 Question 3

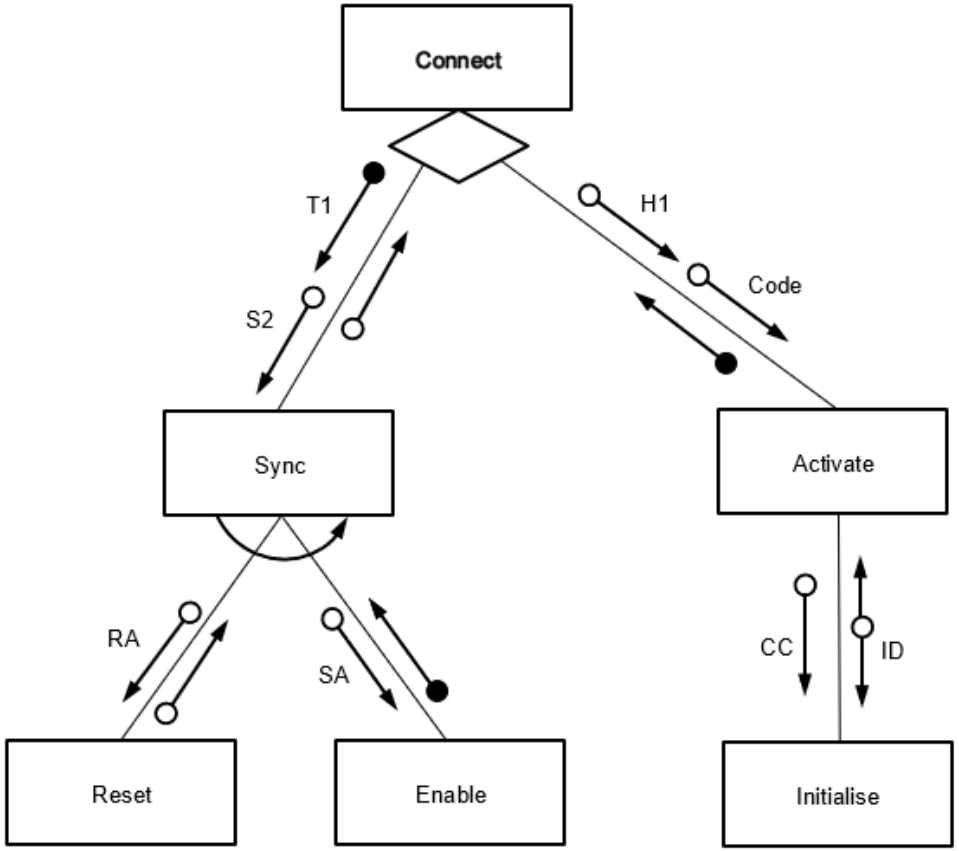
4(a)	One mark per point: **Type:** Run-time error **Cause:** A divide by zero operation is attempted // Attempt to convert a non-numeric string to a number	**2**

Examiner report missing
9618_w24_qp_21 Question 4

5(a)	<table><tr><th>Activity</th><th>Name of life cycle stage</th></tr><tr><td>The walkthrough method is used.</td><td>Testing</td></tr><tr><td>An algorithm is implemented in a programming language.</td><td>Coding</td></tr><tr><td>The client is interviewed about problems with the current system.</td><td>Analysis</td></tr><tr><td>The program is modified to run on new hardware.</td><td>Maintenance</td></tr><tr><td>Records and file structures are defined.</td><td>Design</td></tr></table> One mark per row	**5**
5(b)(i)	One mark per row <table><tr><th>Type of test data</th><th>Test data value</th><th>Expected result</th></tr><tr><td>Abnormal</td><td>12 (< 23)</td><td>FALSE</td></tr><tr><td>Abnormal / Boundary / Extreme</td><td>23</td><td>FALSE</td></tr><tr><td>Boundary / Extreme</td><td>24</td><td>TRUE</td></tr><tr><td>Boundary</td><td>25</td><td>TRUE</td></tr><tr><td>Boundary</td><td>36</td><td>TRUE</td></tr><tr><td>Boundary /Extreme</td><td>37</td><td>TRUE</td></tr><tr><td>Abnormal / Boundary / Extreme</td><td>38</td><td>FALSE</td></tr><tr><td>Abnormal</td><td>99 (> 38)</td><td>FALSE</td></tr></table> **Max 4 marks**	**4**

5(b)(ii)	Integration (testing)	**1**

Examiner report missing
9618_w24_qp_21 Question 5

7(a)	 One mark per point: 1 All module identified in correct hierarchy 2 Parameters to both Sync and Activate **and** return values 3 Parameters to both Reset and Enable 4 Parameters to Initialise 5 Decision diamond	**5**

7(b)	Explanation: Means that Sync repeatedly calls Reset and (then) Enable One mark for each point: MP1: reference to iteration MP2: naming all three modules correctly including correct sequence of calls	**2**

Examiner report missing
9618_w24_qp_21 Question 7

8(b)	Example Solution:
	
	`PROCEDURE CheckAll(CNum : INTEGER)`
	`  DECLARE Index, Count, ThisMark: INTEGER`
	
	`  Count ← 0`
	`  OPENFILE "GRList.txt" FOR WRITE`
	`  FOR Index ← 1 to CNum`
	`  ThisMark ← Result[Index, 1] // 2D array: mark + ID`
	`  IF CheckMark(ThisMark) = TRUE THEN`
	`  WRITE "GRList.txt", NUM_TO_STR(Result[Index, 2])`
	`  Count ← Count + 1`
	`  ENDIF`
	`  NEXT Index`
	`  CLOSEFILE "GRList.txt"`
	`  OUTPUT "There are ", Count, " papers to check"`
	
	`ENDPROCEDURE`
	
	Mark as follows:
	1 Procedure heading, parameter and ending.
	2 Open file in write mode and subsequently close
	3 Loop through all `CNum` candidates
	4 Extract candidate mark from `Result` array **in a loop**
	5 Use of `CheckMark()` with candidate mark as parameter **in a loop**
	6 Test return value and if `TRUE` then increment `Count` **in a loop**
	7 ... write ID to file ...
	8 ... after conversion to a string **in a loop**
	9 Final output of message giving number of papers to check **not in a loop**
	
	Max 8 marks
		8

8(c)	The file will need to be opened in <u>APPEND</u> mode	**1**

Examiner report missing
9618_w24_qp_21 Question 8

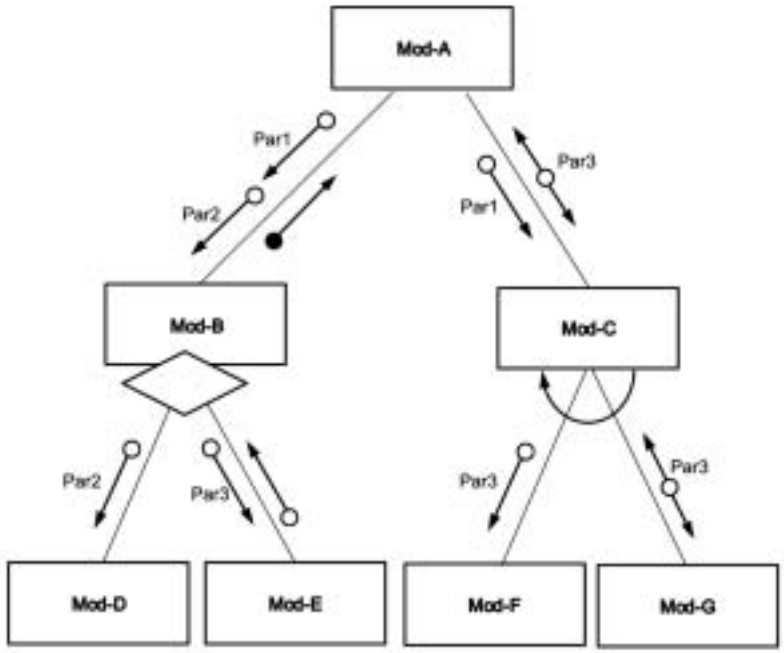
6(a)	Example Solutions:
	
	`PROCEDURE Progress(Percent : INTEGER, Root : STRING)`
	`  DECLARE StepValue : INTEGER`
	`  DECLARE Filename : STRING`
	
	`  StepValue ← (Percent DIV 10) + 1 // INT(Percent / 10) + 1`
	`  FileName ← Root & "-" & NUM_TO_STR(StepValue) & ".bmp"`
	`  CALL Display(Filename)`
	`ENDPROCEDURE`
	
	Alternative:
	
	`PROCEDURE Progress(Percent : INTEGER, Root : STRING)`
	`  DECLARE StepValue : INTEGER`
	`  DECLARE Filename : STRING`
	`  DECLARE Found : BOOLEAN`
	`  StepValue ← 1`
	`  Found ← FALSE`
	`  REPEAT`
	`  IF Percent < StepValue * 10 THEN`
	`  Found ← TRUE`
	`  ENDIF`
	`  StepValue ← StepValue + 1`
	`  UNTIL Found`
	`  Filename ← Root & "-" & NUM_TO_STR(StepValue - 1) & ".bmp"`
	`  CALL Display(Filename)`
	`ENDPROCEDURE`
	
	Mark as follows for use of DIV/INT or loop solution:
	1 Procedure heading, parameters and ending
	2 Calculate `StepValue` as integer value // Attempt at calculating file number
	3 Add 1 to obtain file number // Completely correct file number calculation
	4 Use of `NUM_TO_STR()` to convert file number to string and use
	5 Concatenate `Root`, hyphen, file number and ".bmp" suffix
	6 Call `Display()` with filename as parameter following a reasonable attempt
		6

6(b)(i)	Example answer: `FUNCTION Progress2(Percent, Steps : INTEGER, Root : STRING) RETURNS STRING` One mark for each: • Keyword `FUNCTION Progress2` **and** three parameters of correct type • Keyword `RETURNS` and type string	**2**

6(b)(ii)	The progress display will more accurately show the progress of the task	**1**

Examiner report missing
9618_s24_qp_23 Question 6

7(a)(i)	Mod-B() and Mod-E()	**1**

7(b)	 One mark per bullet: 1 All modules correctly labelled and interconnected 2 Parameters between Mod-A and Mod-B and return value from Mod-B 3 Parameters between Mod-A and Mod-C 4 Diamond applied to Mod-B only 5 Iteration arrow applied to Mod-C only 6 All parameters at lower level and return value	**6**

Examiner report missing
9618_s24_qp_23 Question 7

1(a)		**4**

Pseudocode example	Selection	Iteration	Input/Output

`FOR Index ← 1 TO 10`
`  Data[Index] ← 0`
`NEXT Index`
		✓	

`WRITEFILE ThisFile, "*****"`			✓
`UNTIL Level > 25`		✓	

`IF Mark > 74 THEN`
`  READFILE OldFile, Data`
`ENDIF`
	✓		✓

One mark per row.

1(b)	**Expression**
	`MyInt ← INT (3.1415926)`
	`MyChar ← MID ("Elwood", 3, 1)`
	Any of:
	• `MyString ← NUM_TO_STR (INT (27.509))`
	• `MyString ← CHR (INT (27.509))`
	• `MyString ← TO_UPPER( NUM_TO_STR( 27.509))`
	• `MyString ← TO_LOWER( NUM_TO_STR( 27.509))`
	Any of:
	• `MyInt ← STR_TO_NUM ( RIGHT ("ABC123", 3))`
	• `MyInt ← LENGTH ( RIGHT ("ABC123", 3))`
	• `MyInt ← LENGTH ( LEFT ("ABC123", 3))`
	One mark per row
		4

1(c)	1 mark for stating a suitable way of documenting: • Identifier table 1 mark for giving one piece of information that should be recorded. examples include: Explanation of what (each) variable is used for The purpose of (each) variable An example of data values stored // Initialisation value	**2**

Examiner report missing
9618_s24_qp_22 Question 1

5(a)(i)	One mark per error: Syntax: 1. `NEXT Index` (should be `ENDWHILE)` 2. '&' used to concatenate an integer (in `OUTPUT` statement) Other: 3. Accesses element outside range // Accesses element 0	**3**

5(b)	Run-time	**1**

Examiner report missing
9618_s24_qp_22 Question 5

7(a)	Examples include: Module: `IdentifyMember()` Use: Identifies a club member who has expressed an interest in a given class Module: `GetMemberPhoneNumber()` Use: Gets the mobile phone number of a member Module: `CreateMessage()` Use: Generates a text message to a member Module: `SendMessage()` Use: Sends a text message to a member in the waiting list One mark for name **and** use **Note: max 3 marks**	**3**

7(b)(i)		**5**

Input	Output	Next state
		S1
Input-A	none	S3
Input-A	Output-W	**S3**
Input-B	none	**S2**
Input-B	**none**	**S5**
Input-A	**none**	**S2**
Input-A	**Output-X**	S4

One mark per row 3 to 7

7(b)(ii)	Input-B, Input-A	**1**

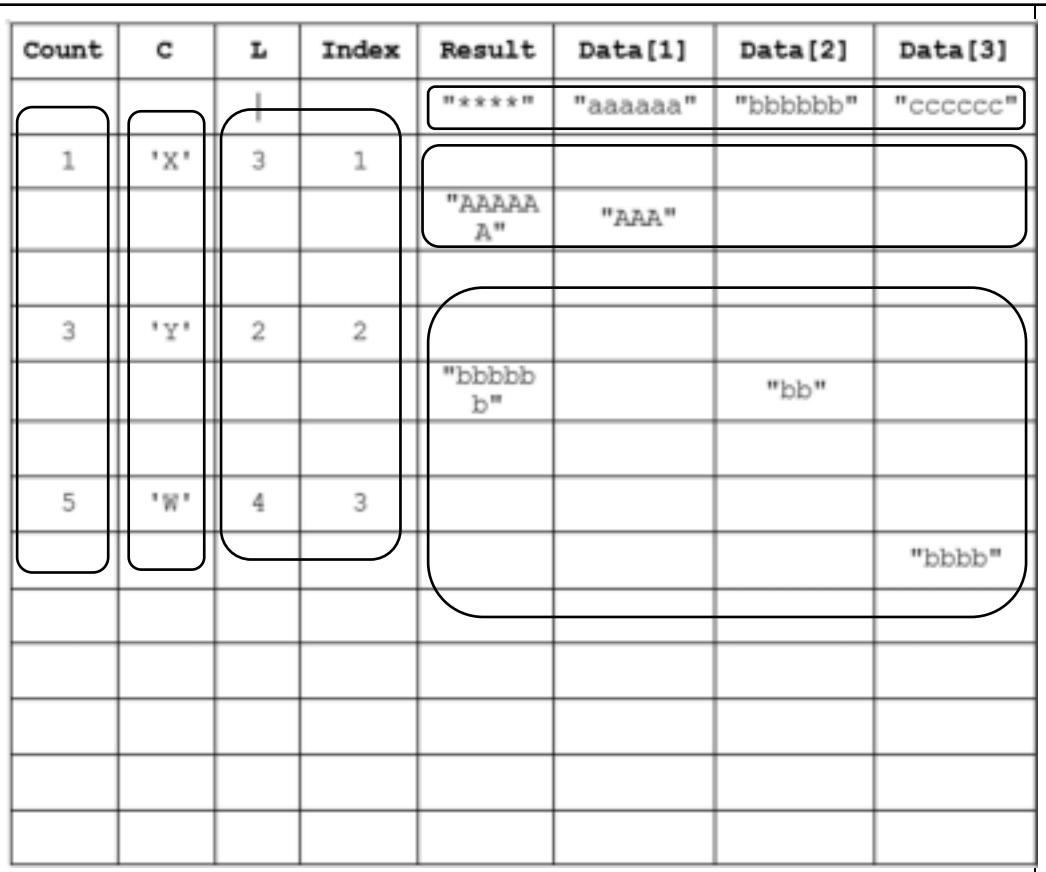
Examiner report missing
9618_s24_qp_22 Question 7

1(a)	<table><tr><th>Assignment statement</th><th>Data type</th></tr><tr><td>A ← LEFT(MyName, 1)</td><td>CHAR / STRING</td></tr><tr><td>B ← Total * 2</td><td>INTEGER / REAL</td></tr><tr><td>C ← INT(ItemCost) / 3</td><td>REAL</td></tr><tr><td>D ← "Odd OR Even"</td><td>STRING</td></tr></table> One mark per row	**4**

1(c)(i)	The names do not reflect / indicate the purpose of the variable // the names are not meaningful	**1**

1(c)(iii)	One mark from: • Indentation / use of white space • Capitalisation of keywords • Use of comments • Use of modular programming • Use of local variables **Note: max 1 mark**	**1**

Examiner report missing
9618_s24_qp_21 Question 1

5(a)	 One mark per zone.	**6**
5(b)(i)	OTHERWISE : CALL Error(C)	**1**
5(b)(ii)	After the 'Z' clause in the CASE construct // before the ENDCASE	**1**

Examiner report missing
9618_s24_qp_21 Question 5

6(a)	 ``` FUNCTION IsRA(x1, y1, x2, y2, x3, y3 : INTEGER) RETURNS BOOLEAN DECLARE Len1, Len2, Len3 : INTEGER Len1 ← (x1 - x2) ^ 2 + (y1 - y2) ^ 2 Len2 ← (x1 - x3) ^ 2 + (y1 - y3) ^ 2 Len3 ← (x2 - x3) ^ 2 + (y2 - y3) ^ 2 IF (Len1 = Len2 + Len3) OR__ (Len2 = Len1 + Len3) OR__ (Len3 = Len1 + Len2) THEN RETURN TRUE ELSE RETURN FALSE ENDIF ENDFUNCTION ``` Mark as follows: 1 Calculate the square of the length of at least one side 2 Calculation of all three lengths squared correctly 3 **One** correct comparison of square of three lengths 4 **All three** comparisons… 5 combined using logical operators / nested `IF` // completely correct selection 6 Return result correctly in both cases	**6**
6(b)	1 mark for statement of problem: Problem: • The function will return an incorrect value // the test will fail 1 mark for solution: Solution: • Round the calculated values (to a known number of decimal places) • Define a threshold below which any difference can be ignored	**2**

Examiner report missing
9618_s24_qp_21 Question 6

7(a)(i)	To make the solution easier to design / implement / solve	**1**

7(b)(i)	Means that `Update` calls (one of) either `Sub-A, Sub-B` or `Sub-C` One mark for each point: • reference to selection / decision / if • naming all four modules correctly	**2**

7(b)(ii)	`PROCEDURE Sub-A (Name : STRING, BYREF P2 : BOOLEAN)`
	
	`FUNCTION Sub-B (P1 : REAL) RETURNS REAL`
	
	One mark per underlined part in each case
		4

Examiner report missing
9618_s24_qp_21 Question 7

1(a)	<table><tr><th>Assignment statement</th><th>Data type</th></tr><tr><td>MyVar1 ← Total1 / Total2</td><td>REAL</td></tr><tr><td>MyVar2 ← 27/10/2023</td><td>DATE</td></tr><tr><td>MyVar3 ← "Sum1 / Sum2"</td><td>STRING</td></tr><tr><td>MyVar4 ← Result1 AND Result2</td><td>BOOLEAN</td></tr></table>	**4**

1(d)	**MP1** Type: Adaptive **MP2** Reason: The (user) requirement(s) changes // to accommodate legislative changes	**2**

Examiner report missing
9618_w23_qp_23 Question 1

5		6

Index	Value	Total	Mix[1]	Mix[2]	Mix[3]	Mix[4]
2		0	4	2	3	5
	2	2				
				5		
3						
	3	5				
					8	
4						
	5	10				
						9
2						
	5	15				
				13		
2						
	13	28				
				21		
2						
			56			

MP1 Row 1 (initialisation)

Each iteration (1 – 5):
MP2 1 – Total 2
MP3 2 – Total 5
MP4 3 – Total 10
MP5 4 – Total 15
MP6 5 – Total 28 **and** final `Mix[1] =` 56

Examiner report missing
9618_w23_qp_23 Question 5

7(a)	**MP1** iteration / looping **MP2** naming all four modules correctly in the correct sequence // e.g. `Module-A` repeatedly calls `Sub-Y1`, then `SubY2` then `Sub-9`	**2**

Examiner report missing
9618_w23_qp_23 Question 7

8(a)	 PROCEDURE ReceiveFile(FileName : STRING) DECLARE FileData : STRING DECLARE CharCount : INTEGER CONSTANT Terminator = "****" OPENFILE FileName FOR WRITE FileData ← GetData() CharCount ← 0 WHILE FileData <> Terminator WRITEFILE FileName, FileData CharCount ← CharCount + LENGTH(FileData) FileData ← GetData() ENDWHILE CLOSEFILE FileName OUTPUT CharCount (, " characters were written to ", FileName) ENDPROCEDURE **MP1** OPEN file in WRITE mode and subsequently CLOSE **MP2** Conditional loop until terminator received **MP3** 'Attempted' use of `GetData()` – Ignore CALL … **MP4** Fully correct use `GetData()` to return the data **in a loop** **MP5** Maintain `CharCount` **in a loop** **MP6** Write each line to the file - **except** the terminator **in a loop** **MP7** Final output of message giving number of characters written **outside loop**	**7**

8(b)	**Max 3** marks Problem: • If the file being sent contains a line of the string "****" • then the file being written by `ReceiveFile()` will end at this point // subsequent file lines will be lost Solution: • Read the file (at the sending end) to find the number of lines it contains • Send an initial message which defines the number of lines in the file ALTERNATIVE SOLUTION: • (Transmitter program) chooses a **different** terminator string / character that doesn't occur in the file • Transmitter program sends the terminator string / character before first line of file / before the transfer begins	**3**

8(c)	 PROCEDURE Chat(Destination : STRING, Port : INTEGER) DECLARE Data : STRING DECLARE Finished : BOOLEAN CONSTANT Terminator = "Bye" CONSTANT STX = CHR(2) CONSTANT ETX = CHR(3) Finished ← FALSE REPEAT Data ← GetData() OUTPUT Data IF Data = Terminator THEN Finished ← TRUE ENDIF IF NOT Finished THEN //about to reply INPUT Data Transmit(STX & Destination & MyID & Data & ETX, Port) IF Data = Terminator THEN Finished ← TRUE ENDIF ENDIF UNTIL Finished = TRUE ENDPROCEDURE Conditional loop **MP1 Conditional loop** **MP2** Test for terminator in both cases **MP3** Use `GetData()` to get the data from the message **MP4** `OUTPUT` the data **in a loop** **MP5** `INPUT` the data reply **MP6** 'Attempted ' use of `Transmit` to send it **in a loop** **MP7** Correct formation of parameters to `Transmit()`	**7**

8(d)	**Note: Max 3** marks **(from either limitation or modification list)** Limitation: 1 `GetData()` does not return a value until a message has been received 2 So once a message has been sent the user has to wait for a reply // chat is half-duplex Modification: 3 If no response allow the receiver to exit chat at any time … 4 `GetData()` should immediately return a suitable message // set a time limit 5 ... which `Chat()` can detect and respond by allowing the conversation to continue	**3**

Examiner report missing
9618_w23_qp_23 Question 8

Examiner report missing
9618_w23_qp_23 Question 52

Examiner report missing
9618_w23_qp_23 Question 53

4(a)	Example Solution
	
	`PROCEDURE Count()`
	`  DECLARE COdd, CEven, ThisNum : INTEGER`
	
	`  COdd ← 0`
	`  CEven ← 0`
	
	`  INPUT ThisNum`
	
	`  WHILE ThisNum <> 99`
	`  IF ThisNum MOD 2 = 1 THEN`
	`  COdd ← COdd + 1`
	`  ELSE`
	`  CEven ← CEven + 1`
	`  ENDIF`
	`  INPUT ThisNum`
	`  ENDWHILE`
	
	`  OUTPUT "Count of odd and even numbers: ", COdd, CEven`
	
	`ENDPROCEDURE`
	
	Mark as follows **Max 6** marks:
	
	1 Procedure heading and ending
	2 **Local** `COdd`, `CEven` and `ThisNum` declared as integers
	3 Conditional loop while `ThisNum <> 99`
	4 Input `ThisNum` **in a loop**
	5 Check `ThisNum` is odd or even **in a loop**
	6 Increment appropriate count **in a loop**, both counts must have been initialised **before loop**
	7 **After the loop** output `COdd` and `CEven` **with a** suitable message following a reasonable attempt at counting
		6

4(b)	**One** mark per set, including stated purpose. **Max 3** marks Example answers: 1 data set with (only) odd values, terminated with 99 2 data set with (only) even values, terminated with 99 3 data sets with same number of odd and even values, terminated with 99 4 data sets with all even / all odd with just one odd/even value, terminated with 99 5 data sets with no values just final 99 6 data sets without (terminating) 99 // missing or incorrectly placed 99	**3**

Examiner report missing
9618_w23_qp_22 Question 4

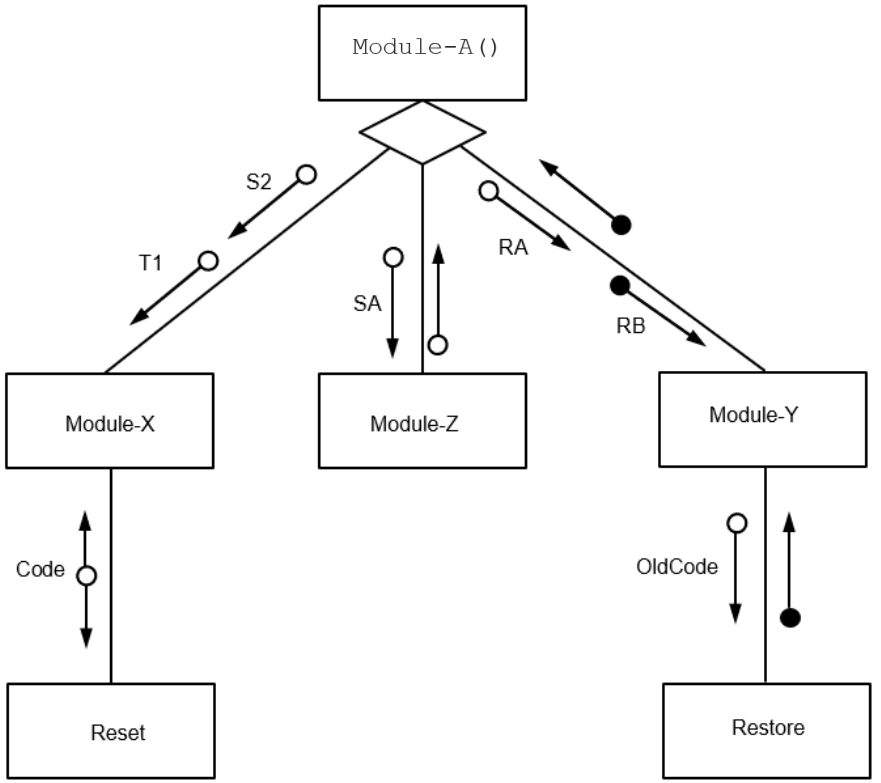
5		6

Index	Value	Count	Mix[1]	Mix[2]	Mix[3]	Mix[4]
3		0	1	3	4	2
	4				3	
4		1				
	2					1
2		2				
	3			2		
3		3				
	3				2	
3		4				
	2				1	
2		5				
						10

One mark for:

- Initialisation row
- Each iteration of Count 1 to 4 with correct `Index`, `Count` and array `Mix` as shown
- Final iteration, Count = 5, correct `Index`, `Count` and array `Mix` as shown plus `Mix[4]` assignment

Examiner report missing
9618_w23_qp_22 Question 5

7(a)	 **One** mark per point: 1 Module names 2 Parameters with labels to Module-X and between Module-X and Reset 3 Parameter (with label) to Module-Z and return from Module-Z 4 Parameters with labels to Module-Y and Restore and return values from Module-Y	**4**
7(b)	Means that `Module-A` calls either one of `Module-X`, `Module-Y` or `Module-Z` (which one is called is decided at runtime). **One** mark for reference to selection **One** mark for naming all **four** modules correctly	**2**

Examiner report missing
9618_w23_qp_22 Question 7

1(a)	**One** mark per row: |	Answer
---	---
The value assigned to `Level` when `ThisValue` is 40	**"Medium"**
The value assigned to Check when `ThisValue` is 36	**12**
The value assigned to Level when `ThisValue` is 18	**"Low"**
The number of elements in array Data that may be incremented	**11**		**4**
1(b)	**One** mark for identifying assignment: **MP1** `Level ← "Very Low"` // the level is assigned value “very low” Explanation points: **MP2** because CASE clauses are checked in sequence // because of the order of the clauses **MP3** a value < 30 satisfies the first clause // Clause '< 20' will never be tested	**3**
1(c)	**MP1** all of the possible values are addressed via all / four / three / the other clauses // there are no other possible values to map to `OTHERWISE`	**1**

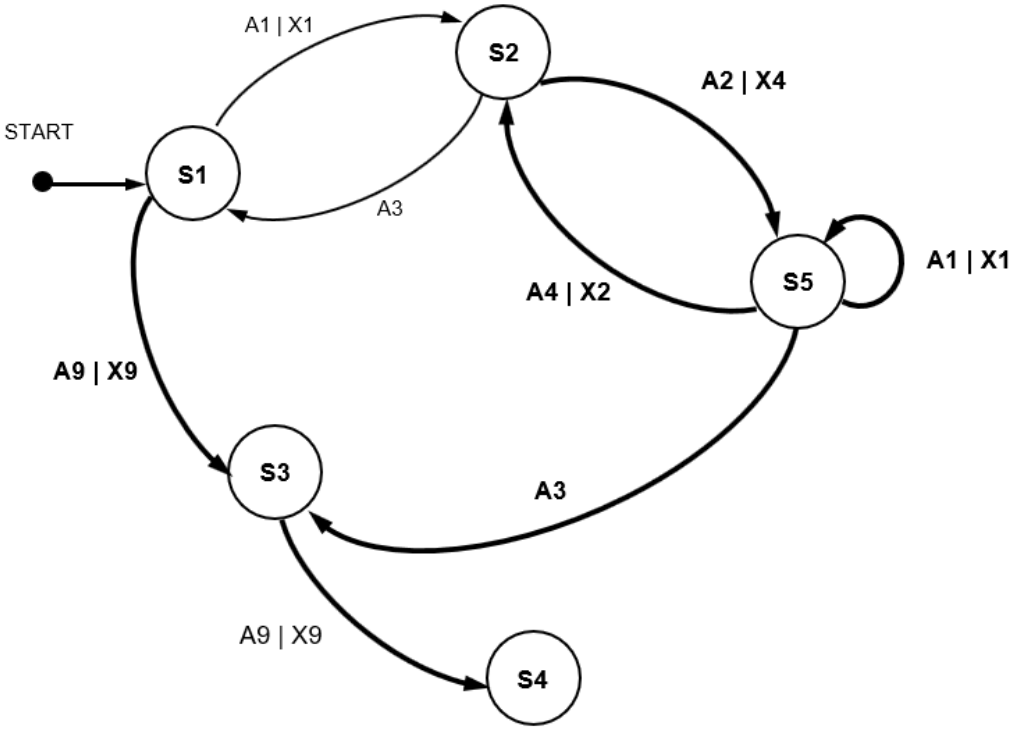
Examiner report missing
9618_w23_qp_21 Question 1

5(a)	**MP1** `Num > Min` should be `Num < Min` **MP2** `Count & 1` should be `Count + 1` **MP3** `NextInput ← "END"` should be `NextInput = "END"`	**3**

5(b)(i)	**MP1** If all the numeric input values are greater than 999 // If there are no numeric values in the sequence **MP2** then the minimum will be given as **999** (and not one of the input values)	**2**

5(b)(ii)	Many possible correct answers, for example: **MP1** Mixture non-numeric and numeric with 3 or 4 values - with all numerics greater than 999 Examples: 1325, DOG, 7868, 7615 // SNAKE, 3478, SPIDER **MP2** Final value: END	**2**

Examiner report missing
9618_w23_qp_21 Question 5

7	 **MP1** S2 labelled **MP2** S3, S4 and S5 added **MP3** Line from S1 to S3 **MP4** All three lines between S2 and S5 (including S5 to S5) **MP5** Line from S5 to S3 **AND** from S3 to S4	**5**

Examiner report missing
9618_w23_qp_21 Question 7

1(a)	**Answer** The dimension of the array: **2** The name of the variable used as an array index: **PCount** The number of elements in the array: **100**	**3**
1(b)	One mark per point: • The (second dimension/index of the) array is declared from 1 to 50 but the loop runs from 0 to 49 • Line number: 10 / 100 / 101 / 102	**2**

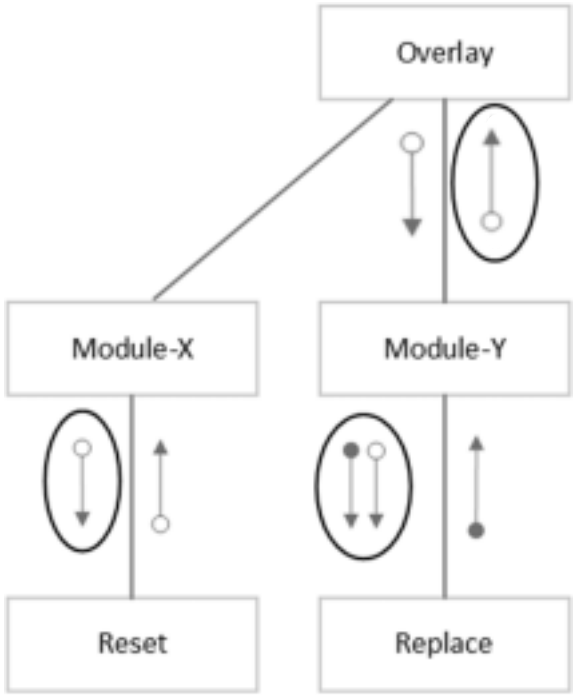
Examiner report missing
9618_s23_qp_23 Question 1

5(a)	**Max 3 marks** Additional Information: MP1 The (program/source) code/specification MP2 test plan // inputs/test data and expected outputs Explanation: MP3 The structure / design / algorithm of the program of the program needs to be known MP4 …. so that all paths through the program can be tested	**3**

5(b)	Perfective	**1**

Examiner report missing
9618_s23_qp_23 Question 5

7(a)(i)	• To filter out information (that is not necessary to solve the problem) // to include only essential information	**1**

7(b)	**Max 3 marks**  MP1 All modules correctly labelled and interconnected. MP2 Correct parameters and return type to Module-X and Reset MP3 Correct parameters and return type to Module-Y and Replace MP4 Correct parameters and return type to Overlay and Module-Y	**3**

Examiner report missing
9618_s23_qp_23 Question 7

8(a)		**8**

```
FUNCTION ChangeSupp(Code1, Code2 : STRING) RETURNS
                                                    INTEGER
  DECLARE Count : INTEGER
  DECLARE ThisLine, ThisCode : STRING

  OPENFILE "Stock.txt" FOR READ
  OPENFILE "NewStock.txt" FOR WRITE
  Count ← 0
  WHILE NOT EOF("Stock.txt")
     READFILE("Stock.txt ", ThisLine) // brackets
                                                    optional
     ThisCode ← MID(ThisLine, 5, 3)
     IF ThisCode = Code1 THEN
        ThisLine ← LEFT(ThisLine, 4) & Code2
                        & RIGHT(ThisLine,
                                  LENGTH(ThisLine) - 7)
        Count ← Count + 1
     ENDIF
     WRITEFILE("NewStock.txt", ThisLine) // brackets
                                                    optional
  ENDWHILE

  CLOSEFILE "NewStock.txt"
  CLOSEFILE "Stock.txt"

RETURN Count
ENDFUNCTION
```

MP1 Open both files, in correct modes, and subsequently close
MP2 Conditional loop until `EOF("Stock.txt")`
MP3 Read a line from `Stock.txt` **AND** extract `ThisCode` **in a loop**
MP4 Test `ThisCode = Code1` AND if true, increment `Count` (must have been Initialised) **in a loop**
MP5 Update `ThisLine` using substring functions and '&' **in a loop**
MP6 completely correct update of `ThisLine` **in a loop**
MP7 Write `ThisLine` to `NewStock.txt` **in a loop**
MP8 Return count after loop

8(b)		**6**

```
PROCEDURE Report_1(Supp : STRING)
  DECLARE Count : INTEGER
  DECLARE ThisItemNum, ThisDesc, ThisLine, ThisCode :
                                                    STRING

  Count ← 0

  OPENFILE "Stock.txt" FOR READ

  OUTPUT "Report for Supplier:" & Supp
  OUTPUT ""           //Blank line as per example
  OUTPUT "Item        Description"
  OUTPUT ""           //Blank line as per example

  WHILE NOT EOF("Stock.txt")
     READFILE("Stock.txt", ThisLine)
     ThisCode ← Mid(ThisLine, 5, 3)
     IF ThisCode = Supp THEN
        ThisItemNum ← LEFT(ThisLine, 4)
        ThisDesc ← RIGHT(ThisLine, LENGTH(ThisLine) - 7)
        OUTPUT ThisItem & "        " & ThisDesc
        Count ← Count + 1
     ENDIF
  ENDWHILE

  CLOSEFILE "Stock.txt"

  OUTPUT "" //Blank line as per example
  OUTPUT "Number of items listed: ", Count
ENDPROCEDURE
```

MP1 Output report header (blank lines optional) – Must contain the parameter code
MP2 Conditional loop until `EOF("Stock.txt")`
MP3 Read a line from `Stock.txt` **AND** extract `SupplierCode` **in a loop**
MP4 Test if `SupplierCode = Supp` then `increment count` (must have been Initialised)
MP5 Extract **AND** output item and description **in a loop**
MP6 Output the final line with count

8(c)(i)	**Max 2 marks** MP1 Must ‘calculate’ the count before any item + description output / after the file is read once MP2 Lines to be output have to be stored … MP3 The file has to be read twice	**2**

8(c)(ii)	One mark per point: MP1 Loop through the file calculating the count MP2 Save ‘selected’ items in an array MP3 (After all lines have been read), output the header lines / count MP4 Loop through the array to output each array element	**3**

Examiner report missing
9618_s23_qp_23 Question 8

1(a)(i)	Use of constants	**1**

1(a)(ii)	One mark per bullet point (or equivalent to max 3): 1 Postal rates are entered once only 2 Avoids input error / changing the cost accidentally // avoids different values for postal rates at different points in the program 3 When required, the constant representing the postal rate value is changed once only // easier to maintain the program when the postal rates change 4 Makes the program easier to understand **Note: Max 3 marks**	**3**

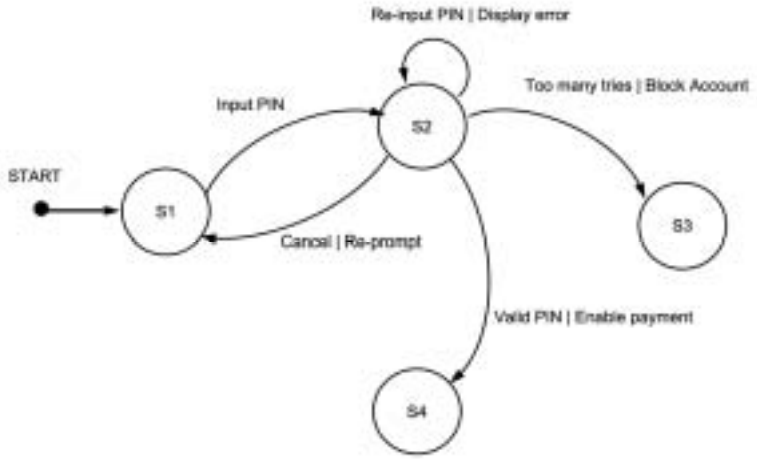
1(b)	One mark per bullet point: • Indentation • White space • Comments • Sensible / meaningful variable names // use of Camel Case • Capitalised keywords **Note: Max 3 marks**	**3**

Examiner report missing
9618_s23_qp_22 Question 1

5(a)	One mark per point: • parameter / `Num` has been passed by reference // should have been passed by value • so when the value / `ThisNum` is modified (in procedure `DisplaySqrt()`) • the new value will be used in the loop (lines 40–43) // `Num` will be changed to modified value	**3**
5(b)	• The rules of the language have not been broken // there are no syntax errors	**1**
5(c)	Could use an IDE to: • Set a breakpoint to stop the program at a certain line / statement / point • Step through the program line by line / statement by statement • checking the value of `'num'` / a variable using a report / watch window One mark per bullet	**3**
5(d)	Answers include: • Change the statement into a comment • Change the statement to a string representing a literal value and assign it to a variable / output it **Note: max 1 mark**	**1**

Examiner report missing
9618_s23_qp_22 Question 5

7(a)(i)	Correct answers include: • Information: customer name Justification: to personalise / address the email • Information: email address Justification: so that the email can be delivered correctly • Information: product category preference Justification: to check that the customer would be interested in the product • Information: contact preference Justification: to check that the customer will accept contact via email • Information: order history Justification: to send details of product similar to that already bought // to identify frequent shoppers • Information: new product information Justification: to include information about the new product in the email One mark for each piece of information **and** matching justification **Note: Max 3 marks**	**3**

7(b)	 One mark for each: 1 Line from S1 to S2 **and** label 2 S2 loop label 3 S3 circle and label on incoming 4 S4 circle and label on incoming	**4**

Examiner report missing
9618_s23_qp_22 Question 7

8(a)	 FUNCTION CheckInfo(NewLine: STRING) RETURNS BOOLEAN DECLARE ThisNum : STRING DECLARE Index : INTEGER IF LENGTH(NewLine) < 19 THEN RETURN FALSE ENDIF FOR Index ← 1 TO 4 IF NOT IS_NUM(MID(NewLine, Index, 1)) THEN RETURN FALSE ENDIF NEXT Index ThisNum ← LEFT(Newline, 4) IF ThisNum < "0001" OR ThisNum > "5999" THEN RETURN FALSE ENDIF IF NOT OnlyAlpha(MID(Newline, 5, 3)) THEN RETURN FALSE ENDIF RETURN TRUE ENDFUNCTION Mark as follows: 1 Test length of parameter 2 Extract first 4 characters of parameter (as ItemNum) 3 Test first four characters are **all** numeric 4 Test ItemNum in range "0001" to "5999" 5 Extract characters 5 to 7 of parameter (as SupplierCode) 6 Use of OnlyAlpha() with extracted SupplierCode 7 Return BOOLEAN value correctly in all cases, must have been declared as local	**7**

8(c)	One mark for method: Method: Stub testing Two marks for description: • The modules `SuppExists()` and `CheckSupplier()` are replaced by dummy modules • ...which return a known result / contain an output statement to show they have been called	**3**

Examiner report missing
9618_s23_qp_22 Question 8

1(a)	For example: Could set a breakpoint to stop the program at a particular point / instruction then the value of variables could be checked using a report/watch window while single stepping can be used to execute one statement/line at a time. Marks available as follows: • One mark for each underlined term • One mark for an explanation of each term **Note: max 4 marks**	**4**

1(b)	One mark for correct description of: Error 1: Brackets mismatch // 2/value should be added after brackets/function // Addition between a string and a number is not valid // STR_TO_NUM / the function needs to be passed a string / not an integer Error 2: No Error Error 3: `MONTH()` returns an integer and this is being compared with a character/string // Integer cannot be compared to a string // 6 should not be in quotes	**3**

Examiner report missing
9618_s23_qp_21 Question 1

5(a)(i)	Reasons include: 1 No working software until late in the life cycle so slower to market than competitors // Does not allow the creation of early versions/prototypes (which can be updated later) 2 More difficult/slower to cope with changes to the requirements // website slower to be updated to reflect new requirements 3 Needs high involvement/feedback of the stake holders /customer / client One mark per point **Max 2 marks**	**2**

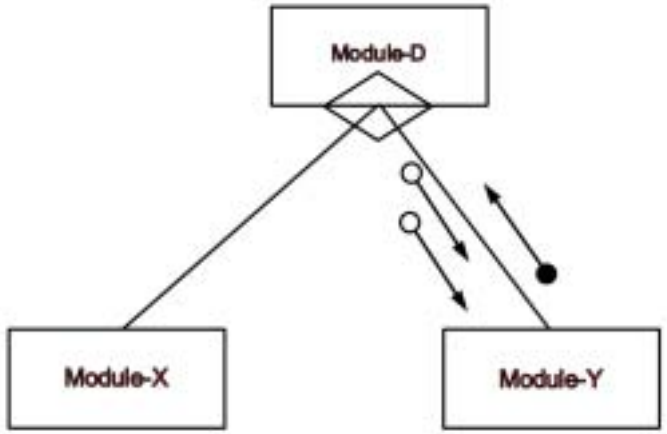
5(a)(ii)	Iterative / Rapid Application Development / RAD	**1**

5(b)	One mark for Stage Stage: Beta testing **Max 2 marks for Description** Description: 1 Testing carried out by a small group of (potential) users 2 Users will check that the website/software works as required / works in the real world //User will identify errors in the website/software 3 Users will feedback (problems) / suggestions for improvement 4 Problems / suggestions identified are addressed (before the program is sold)	**3**

Examiner report missing
9618_s23_qp_21 Question 5

7(a)	Examples include: Module: `GetOverdueLoan()` Use: Identifies an overdue book Module: `IdentifyStudent()` Use: Identifies a student (with an overdue book)1 Module: `GetStudentEmail()` Use: Gets the email address of a student with an overdue book Module: `CreateEmail()` Use: Generates an email to a student with an overdue book Module: `SendEmail()` Use: Sends an email to a student with an overdue book One mark for name **and** use **Note: Max 3 marks**	**3**

7(b)(i)	One mark per point: • Module-A calls the other three modules • The process is repeated	**2**

7(b)(ii)	 One mark per bullet point: • All rectangles correctly labelled and interconnected • All and only parameters as shown • Selection diamond	**3**

Examiner report missing
9618_s23_qp_21 Question 7

8(a)	 ``` FUNCTION IsNewSupp(ThisString : STRING) RETURNS BOOLEAN DECLARE Index : INTEGER DECLARE ThisChar : CHAR IF LENGTH(ThisString) <> 5 THEN RETURN FALSE // invalid SupplierCode length ENDIF IF SuppExists(ThisString) THEN RETURN FALSE // SupplierCode already exists ENDIF FOR Index ← 1 TO 5 ThisChar ← TO_LOWER(MID(ThisString, Index, 1)) IF ThisChar < 'a' OR ThisChar > 'z'THEN RETURN FALSE ENDIF NEXT Index RETURN TRUE ENDFUNCTION ``` Mark as follows: 1 Check `ThisString` is exactly 5 characters in length 2 Use of `SuppExists()` with a string of 5 characters as a parameter 3 Loop for 5 iterations // loops for length of string parameter 4 Extract a character **in a loop** 5 Test if char is alphabetic **in a loop** 6 **…**catering for both upper and lower case 7 Return Boolean following a reasonable attempt	**7**

8(b)	
	``
	FUNCTION CheckNewItem(NewLine : STRING) RETURNS BOOLEAN
	  DECLARE NotFound : BOOLEAN
	  DECLARE NewItemNum, ThisItemNum, ThisLine : STRING
	
	  NotFound ← TRUE
	
	  OPENFILE "Stock.txt" FOR READ
	  NewItemNum ← LEFT(NewLine, 4)
	  ThisItemNum ← "0000" //rogue initial value
	
	  WHILE NOT EOF("Stock.txt") AND NotFound = TRUE AND__
	                                ThisItemNum < NewItemNum
	    READFILE("Stock.txt", ThisLine) //brackets optional
	    ThisItemNum ← LEFT(ThisLine, 4)
	    IF ThisItemNum = NewItemNum THEN
	      NotFound ← FALSE
	    ENDIF
	  ENDWHILE
	
	  CLOSEFILE "Stock.txt"
	
	  RETURN NotFound
	ENDFUNCTION
	``
	
	Mark as follows:
	
	1 Open `Stock.txt` in READ mode and subsequently close
	2 Extract `NewItemNum` from parameter
	3 Conditional loop until `EOF("Stock.txt")`
	4 ... **OR** `NewItemNum` found
	5 ... **OR** when `ThisItemNum < NewItemNum`
	6 Read a line from `Stock.txt` **AND** extract `ThisItemNum` **in a loop**
	7 If `ThisItemNum = NewItemNum` then terminate loop / set flag **in a loop**
	8 Return Boolean after reasonable attempt
	
	Max 7 marks
	
	Max 6 if function wrapper (heading and ending) missing or incorrect
		7

8(c)(i)	Integration testing	**1**

8(c)(ii)	Two marks for the description: • A dummy/simple module is written to replace the module that does not work properly • The dummy/simple module will return an expected value // will output a message to show it has been called	**2**

Examiner report missing
9618_s23_qp_21 Question 8

<table>
<tr><td>1(a)</td><td>One mark per row<table><tr><th>Variable use</th><th>Data type</th></tr><tr><td>Store the number of days in the current month</td><td>INTEGER</td></tr><tr><td>Store the first letter of the customer's first name</td><td>CHAR</td></tr><tr><td>Store an indication of whether a year is a leap year</td><td>BOOLEAN</td></tr><tr><td>Store the average amount spent per customer visit</td><td>REAL</td></tr></table></td><td>4</td></tr>
<tr><td>1(b)(i)</td><td>Easier to manage/plan/cost // Clear deliverables produced at (end of) each stage</td><td>1</td></tr>
<tr><td>1(b)(ii)</td><td>One mark per point (Max 1):

• The problem definition
• Requirements specification // Client requirements
• Documentation related to current system (e.g. ER diagram of current system, DFD of current system, feasibility study)</td><td>1</td></tr>
<tr><td>1(c)(i)</td><td>One mark per point (Max 1):

• Modules are developed in parallel / as prototypes
• Minimal / no detailed planning is carried out // Allows for changes to requirements
• Flexible development process
• Small incremental releases are made, each adding functionality
• Used for time critical development
• Client involved during (all stages) of development</td><td>1</td></tr>
</table>

1(c)(ii)	Examples include: Benefits: **(Max 2 marks)** • Quicker development possible / Multiple areas can be worked on at same time • Prototype produced (at early stage in process) • Easier to change requirements / quicker delivery of usable modules • Early review possible / closer cooperation between client and developers Drawback: **(Max 1 mark)** • Difficult to estimate cost / time to complete project • Documentation often omitted • Lack of client availability throughout life cycle // too easy for client to keep changing their mind	**3**

1(d)	One mark for each point (**Max 2**) Examples include: 1 Change to website requirements 2 New technologies available to host website // changes made to library modules used 3 Change in relevant legislation	**2**

Examiner report missing
9618_w22_qp_23 Question 1

5(a)(i)	One mark per point:
	
	1 Procedure heading and ending including four parameters...
	2 ...and use of `BYREF` for the three extracted values
	3 Extract and assign `SID`
	4 Extract and assign `SDesc`
	5 Calculation of length of `SCost` (remainder of string)
	6 Extract and assign `SCost` following an attempt at MP5
	
	`PROCEDURE UnPack(BYVAL TLine : STRING, BYREF SID, SDesc, SCost : STRING)`
	`  SID ← LEFT(TLine, 5)`
	`  SDesc ← MID(TLine, 6, 32)`
	`  SCost ← RIGHT(TLine, LENGTH(TLine) - 37)`
	`ENDPROCEDURE`
		6

5(c)	One mark for reason One mark for each example (**Max 2**) Reason: • Makes the code easier to understand // Describes the purpose of the identifier // Makes the code easier to debug/test/maintain Further examples include: • White space • Indentation • Keywords in capitals • Comments • Local variables // parameters	**3**

5(d)	One mark per point (**Max 3**) 1 The program is checked by creating a trace table / going through the program a line at a time 2 ….to record/check variable (values) as they change 3 Error may be indicated when variable given an unexpected value 4 Error may be indicated by an unexpected path through the program // Faults in the logic of the program can be detected	**3**

Examiner report missing
9618_w22_qp_23 Question 5

6(a)	One mark per row Examples:	**5**

Test number	Component weight	Min	Max	Expected return value
1	300	290	315	'A'
2	> 317	290	315	'R'
3	317	290	315	'C'
4	288	290	315	'C'
5	< 288	290	315	'R'

Examiner report missing
9618_w22_qp_23 Question 6

1(a)(i)	One mark for each point (**Max 2**): • When a task which is repeated / reused / performed in several places • When a part of an algorithm performs a specific task • Reduces complexity of program / program is simplified // subroutine already available • Testing / debugging / maintenance is easier	**2**

1(b)	One mark for test stage, one mark for each description point **(Max 3 for Description)** **Test stage**: Beta testing **Description**: 1 Testing carried out by a small group of (potential) users 2 Users will check that the software works as required / works in the real world / does not contain errors 3 Users will feedback problems / suggestions for improvement 4 Problems / suggestions identified are addressed (before the program is sold)	**4**

Examiner report missing
9618_w22_qp_22 Question 1

2(a)(i)	One mark per step (or equivalent): 1 Open file in APPEND mode (and subsequent Close) 2 Prompt and Input a student name and mark 3 If mark greater than or equal to 20 jump to step 5 4 Write only the name to the file 5 Repeat from Step 2 for 35 times / the number of students	**5**

2(b)	One mark per row (row 2 to 5): **Input** / **Output** / **Next state** ( ) / ( ) / S1 Input-A / **Output-X** / **S2** **Input-A** / **(none)** / S2 **Input-B** / Output-W / **S3** **Input-A** / Output-W / **S4**	**4**

Examiner report missing
9618_w22_qp_22 Question 2

6(a)	Example by repeated multiplication:
	
	Mark as follows (multiplication solution), (**Max 7**):
	
	1 Function heading and ending **including** parameter and return type
	2 Declaration and initialisation of local `Num`
	3 Any conditional loop
	4 Conditional loop until `ThisValue` found or `Try` out of range
	5 Multiply `Try` by `Num` **in a loop**
	6 Compare `Try` with `ThisValue` **and** set termination if the same **in a loop**
	7 Increment `Num` and repeat **in a loop**
	8 Attempt to Return `Num` if `ThisValue` is a factorial or -1 otherwise
	
	`FUNCTION FindBaseNumber(ThisValue : INTEGER) RETURNS INTEGER`
	`  DECLARE Num, Try : INTEGER`
	`  DECLARE Found : BOOLEAN`
	
	`  Num ← 0`
	`  Found ← FALSE`
	`  Try ← 1`
	
	`   WHILE Try <= ThisValue AND Found = FALSE`
	`      Num ← Num + 1`
	`      Try ← Try * Num`
	`      IF Try = ThisValue THEN //BaseNumber found`
	`         Found ← TRUE`
	`      ENDIF`
	`   ENDWHILE`
	
	`   IF Found = TRUE THEN`
	`      RETURN Num`
	`   ELSE`
	`      RETURN -1`
	`   ENDIF`
	
	`ENDFUNCTION`
		7

6(b)	One mark per row. Examples of invalid strings: 1 Non-numeric but not "End" // contains space or other non-numeric characters 2 Real number 3 Integer value out of range (i.e. <= 0) 4 Empty string 5 Correct word but wrong case e.g. "end" rather than "End"	**4**

Input	Reason for choice
"Aardvark"	Non-numeric (and not "End")
"27.3"	Numeric but not an integer
"-3" // "0"	A non-positive integer
""	An empty string

Examiner report missing
9618_w22_qp_22 Question 6

1(a)	One mark per point: 1 They are tried and tested so free from errors 2 They perform a function that you may not be able to program yourself (for example **encryption**) 3 They are readily available / speed up development time	**3**

1(c)	One mark per row: **Statement** / **Error** `IF EMPTY ← "" THEN` — Should be "=" not ← `Status ← IS_NUM(-23.4)` — Parameter should be a string (or char) // should not be a real `X ← STR_TO_NUM("37") + 5` — NO ERROR `Y ← STR_TO_NUM("37" + "5")` — Wrong operator – should be & or Parameter is not a string	**4**

Examiner report missing
9618_w22_qp_21 Question 1

6(a)	One mark for **any** part correct (accept equivalent wording) (**Max 1**): • Condition evaluates to TRUE if bracket contents evaluate to FALSE: • Bracket contents evaluate to FALSE if: • Dots: zero / less than one or • Ats: not equal to one or • Others: less than nine	**1**

6(b)(i)	One mark for each area as outlined:	**5**

Index	NextChar	Dots	Ats	Others	Valid
		0	0	0	TRUE
1	'L'			1	
2	'i'			2	
3	'z'			3	
4	'.'	1			
5	'1'			4	
6	'2'			5	
7	'3'			6	
8	'@'		1		
9	'b'			7	
10	'i'			8	
11	'g'			9	
12	'@'		2		FALSE

6(b)(ii)	FALSE	**1**

Examiner report missing
9618_w22_qp_21 Question 6

7(a)(i)	One mark per point (**Max 7**) as follows:
	
	1 Declaration of local variables for `Par1` `Par2` and `Par3`
	2 Loop to end of (parameter) string // until operator is found
	3 Extract a character **in a loop**...
	4 Attempt at extraction of three parts of expression using substring functions
	5 Completely correct extraction of all three parts of expression
	6. Convert string to Integer using `STR_TO_NUM(<something sensible>)`
	7 Attempt to interpret at least two operators (`Par2`): + - * /
	8 Corresponding correct calculation (all operators) **and** final Output of result
	
	`PROCEDURE Calculate(Expression : STRING)`
	`  DECLARE Val1, Val2, Index : INTEGER`
	`  DECLARE Result : REAL`
	`  DECLARE Par1, Par2, Par3 : STRING`
	
	`  CONSTANT PLUS = '+'`
	`  CONSTANT MINUS = '-'`
	`  CONSTANT MULTIPLY = '*'`
	`  CONSTANT DIVIDE = '/'`
	
	`  FOR Index ← 1 TO LENGTH(Expression) //search for operator`
	`  ThisChar ← MID(Expression, Index, 1)`
	`  IF IS_NUM(ThisChar) = FALSE THEN`
	`  Par1 ← LEFT(Expression, Index - 1)`
	`  Par2 ← ThisChar`
	`  Par3 ← RIGHT(Expression, LENGTH(Expression) - Index)`
	`  ENDIF`
	`  NEXT Index`
	
	`  Val1 ← STR_TO_NUM(Par1)`
	`  Val2 ← STR_TO_NUM(Par3)`
	
	`  CASE OF Par2`
	`  PLUS : Result ← Val1 + Val2`
	`  MINUS : Result ← Val1 - Val2`
	`  MULTIPLY : Result ← Val1 * Val2`
	`  DIVIDE : Result ← Val1 / Val2`
	`  ENDCASE`
	
	`  OUTPUT Result`
	
	`ENDPROCEDURE`
		7

7(b)	**Example string**: "23/0" (Any divide by zero example) **Reason**: The result is infinity / cannot be represented / is undefined // will cause the program to crash	**2**

Examiner report missing
9618_w22_qp_21 Question 7

8(a)	One mark for each point (**Max 7**) as follows: 1 Function heading and ending **including** parameter and return type 2 Declaration **and** initialisation of local Integer for `Count` 3 OPEN in READ mode and CLOSE 4 Conditional loop until `EOF()` 5 Read a line **in a loop** 6 If non-blank, increment count **in a loop** 7 Terminate loop when 10 non-blank lines have been read 8 Return Boolean in both cases	**7**

```
FUNCTION CheckFile(Thisfile : STRING) RETURNS BOOLEAN
   DECLARE Valid : BOOLEAN
   DECLARE ThisLine : STRING
   DECLARE Count : INTEGER

   Count ← 0
   Valid ← FALSE
   OPEN ThisFile FOR READ

   WHILE NOT EOF(ThisFile) AND Valid = FALSE
      READFILE ThisFile, ThisLine
      IF ThisLine <> "" THEN
         Count ← Count + 1
         IF Count > 9 THEN
            Valid ← TRUE
         ENDIF
      ENDIF
   ENDWHILE

   CLOSEFILE ThisFile
   RETURN Valid

ENDFUNCTION
```

8(b)	`CALL CountErrors("Jim01Prog.txt", 20)` One mark for each: 1 Module name, at least one parameter in brackets and one parameter correct 2 Completely correct statement	**2**

8(d)	One mark for each (**Max 2**): Examples: 1 Incorrect block structure. Missing keyword denoting part of block (for example `ENDPROCEDURE, ENDFUNCTION, ENDTYPE`) 2 Data type errors, for example, assigning an integer value to a string 3 Identifier used before it is declared 4 Incorrect parameter use	**2**

Examiner report missing
9618_w22_qp_21 Question 8

1(a)(i)	One from: • The program obeys the rules / grammar of the programming language used • The program will run // it can be compiled / interpreted • Accept by example. e.g. 'no mis-spelt keywords' / 'all brackets match'	**1**

1(a)(ii)	One mark for type plus one for corresponding description Type of error: A logic error Description: • An error in the algorithm / design of the solution • the program does not behave as expected / give the expected output. • Accept by example e.g. wrong arithmetic operator used / wrong loop count **OR** Type of error: Run-time error Description: • The program performs an illegal instruction / invalid operation • Accept by example: divide by zero or endless loop or simply 'crashes' / freezes	**2**

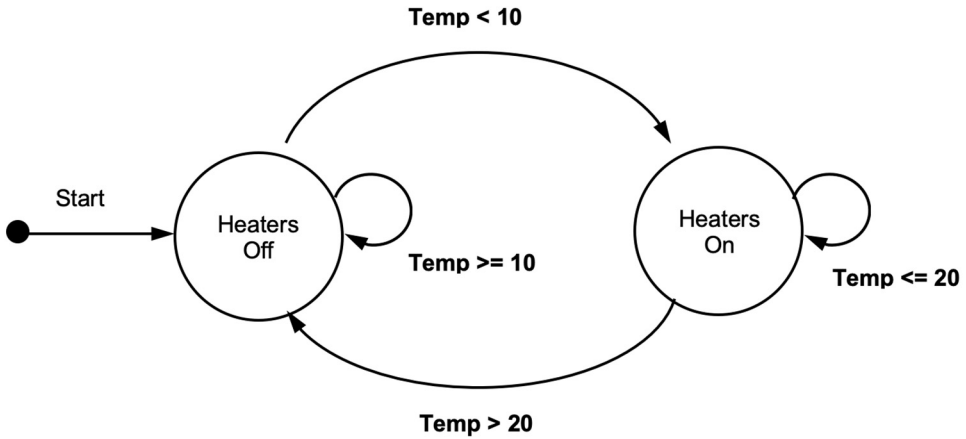
Examiner report missing
9618_w21_qp_23 Question 1

4(a)		**4**

Test	Test data value	Explanation	Expected Outcome
1	23	Normal Data	Data is accepted
2	**0**	**Boundary Data**	**Data is accepted**
3	**40**	**Boundary Data**	**Data is accepted**
4	**>= 41**	**Abnormal Data**	**Data is rejected**
5	**<= −1**	**Abnormal Data**	**Data is rejected**

One mark per row for rows 2 to 5.

4(b)		**3**



One mark for each label:

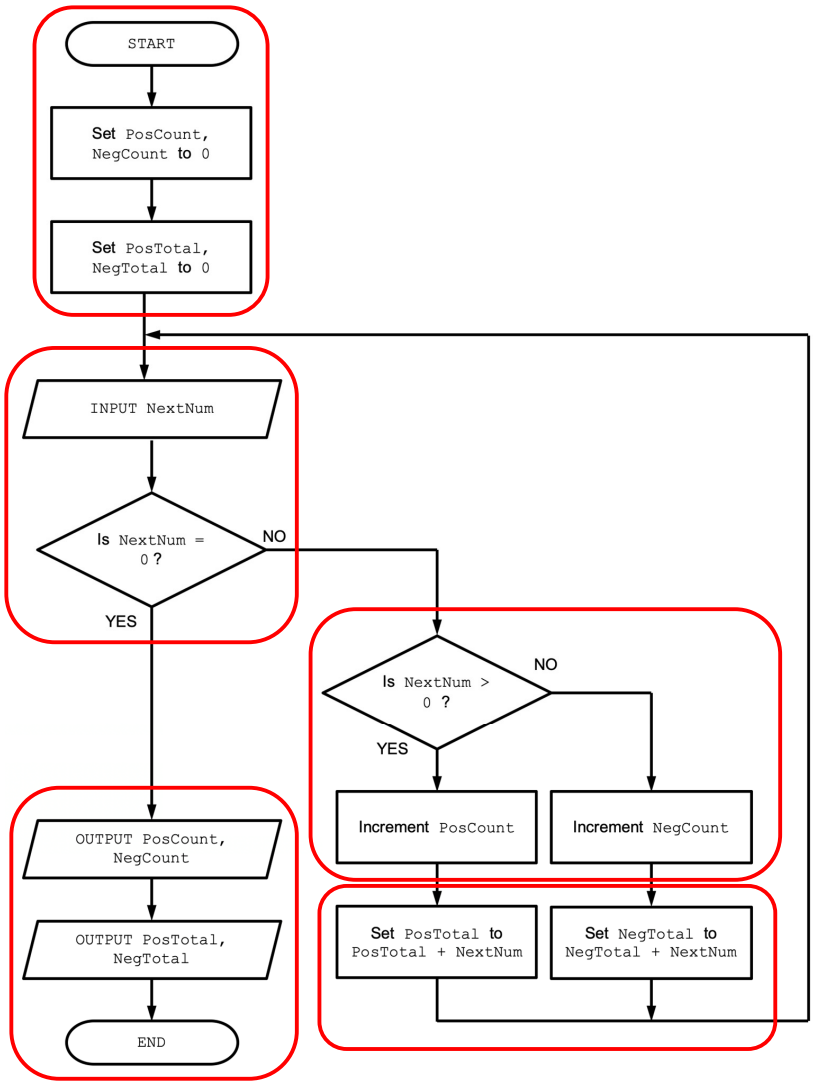
- Temp < 10 (Heaters Off to Heaters On)
- Temp > 20 (Heaters On to Heaters Off)
- on **BOTH** loops (non-contradictory values)

Examiner report missing
9618_w21_qp_23 Question 4

5(a)	One mark for the character and one for the corresponding reason. • Character: Any except alphabetic, numeric, ',' ':' or space • Reason: character doesn't occur in data to be recorded	**2**

5(b)	Design	**1**

Examiner report missing
9618_w21_qp_23 Question 5

2(a)	 One mark for each outlined group. Note: • Sum and increment steps (bottom-right rectangles) may be in reverse order in which case sum group will have two output lines • Max 4 for non-working solution	**5**

2(b)(i)	One mark for each: Life cycle method: Iterative // Rapid Application Development (RAD) Reason: Provides a working model / prototype at an early stage for the principal to approve / review	**2**

2(b)(ii)	Decisions will be made regarding: • Data structures • Algorithms / flowcharts / pseudocode • Program structure (modules) / use of library routines / module - team allocation • User interface // Web-page layout / content (for given scenario) • Testing method / plan • Choice of programming language / program environment **Max 3**	**3**

Examiner report missing
9618_w21_qp_22 Question 2

4(a)		**5**

	Answer
A line number containing a variable being incremented	19 / 21 / 23
The type of loop structure	pre-condition
The number of functions used	3
The number of parameters passed to function `STR_TO_NUM()`	1
The name of a procedure other than `Check()`	Result

4(b)	One mark per point: • Meaningful variable names • Indentation / white space / blank lines • Capitalisation of keywords	**3**

Examiner report missing
9618_w21_qp_22 Question 4

1(a)(i)	One from: • The program obeys the rules / grammar of the programming language used • The program will run // it can be compiled / interpreted • Accept by example. e.g. 'no mis-spelt keywords' / 'all brackets match'	**1**

1(a)(ii)	One mark for type plus one for corresponding description Type of error: A logic error Description: • An error in the algorithm / design of the solution • the program does not behave as expected / give the expected output. • Accept by example e.g. wrong arithmetic operator used / wrong loop count **OR** Type of error: Run-time error Description: • The program performs an illegal instruction / invalid operation • Accept by example: divide by zero or endless loop or simply 'crashes' / freezes	**2**

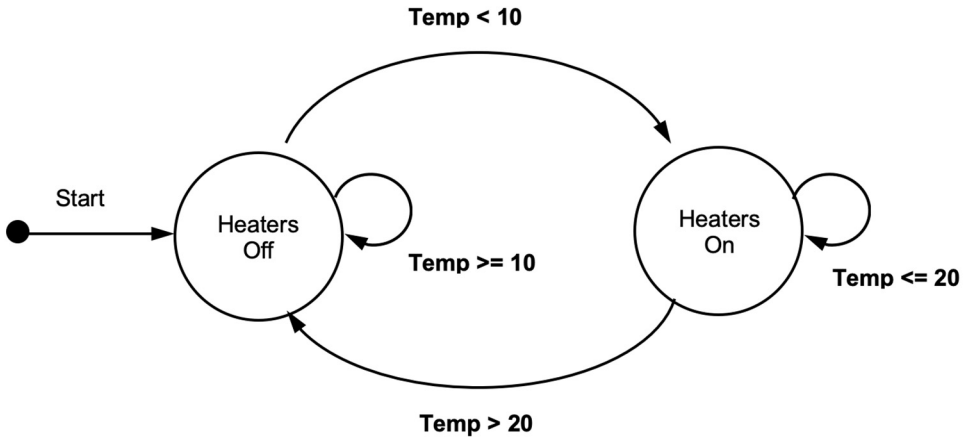
Examiner report missing
9618_w21_qp_21 Question 1

4(a)		**4**

Test	Test data value	Explanation	Expected Outcome
1	23	Normal Data	Data is accepted
2	**0**	**Boundary Data**	**Data is accepted**
3	**40**	**Boundary Data**	**Data is accepted**
4	**>= 41**	**Abnormal Data**	**Data is rejected**
5	**<= −1**	**Abnormal Data**	**Data is rejected**

One mark per row for rows 2 to 5.

4(b)		**3**



One mark for each label:

- Temp < 10 (Heaters Off to Heaters On)
- Temp > 20 (Heaters On to Heaters Off)
- on **BOTH** loops (non-contradictory values)

Examiner report missing
9618_w21_qp_21 Question 4

5(a)	One mark for the character and one for the corresponding reason. • Character: Any except alphabetic, numeric, ',' ':' or space • Reason: character doesn't occur in data to be recorded	**2**

5(b)	Design	**1**

Examiner report missing
9618_w21_qp_21 Question 5

1(a)		**4**

Example value	Explanation	Variable name	Data type
`"Wong"`	The preferred name of the member joining the football club	**`MemberName`**	**`STRING`**
`FALSE`	A value to indicate whether an existing member of the club lives at the same address	**`FamilyMember`**	**`BOOLEAN`**
`19/02/1983`	When the member joined the football club	**`StartDate`**	**`DATE`**
`1345`	The number of points a member has earned. Members of the club earn points for different activities.	**`Points`**	**`INTEGER`**

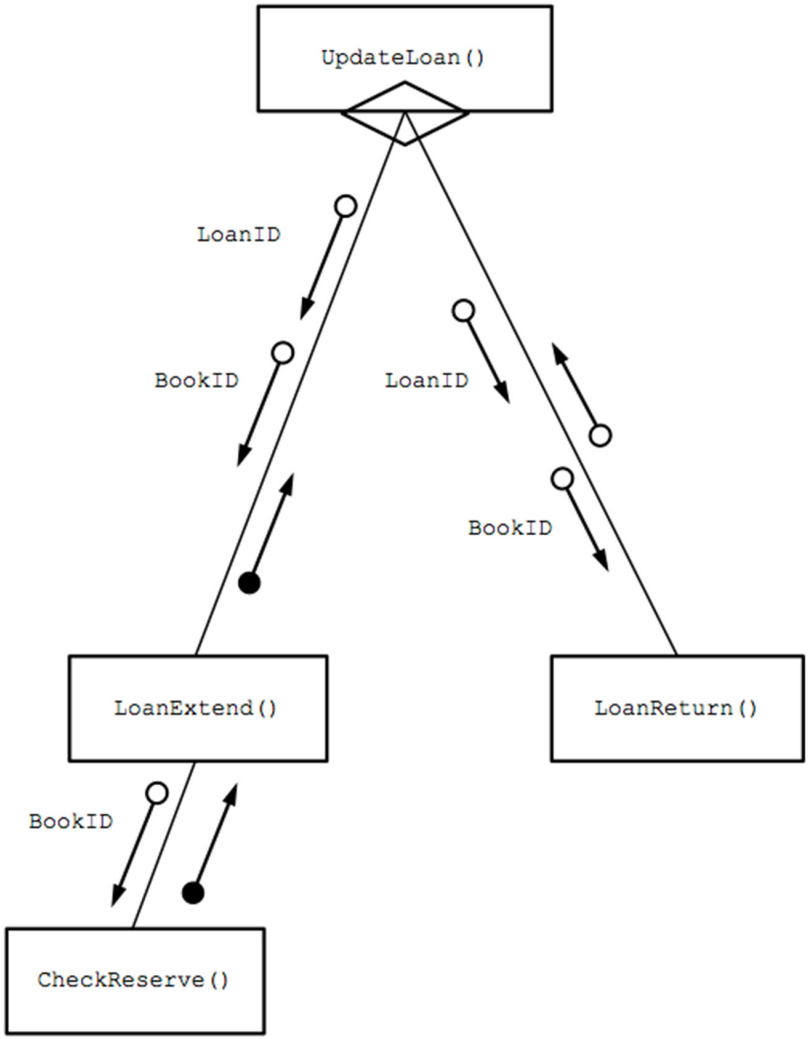
One mark for each appropriate variable name plus data type

1(b)		**5**

Statement	Error
`Result ← 2 & 4`	Should be arithmetic operator (not &) // 2 and 4 should be CHAR / STRING
`SubString ← MID("pseudocode", 4, 1)`	NO ERROR
`IF x = 3 OR 4 THEN`	Not Boolean values / incorrect operator // Condition incorrect
`Result ← Status AND INT(X/2)`	`INT(X/2)` doesn't evaluate to a boolean value / incorrect operator
`Message ← "Done" + LENGTH(MyString)`	Can't add string to number / `"Done"` is not a number

One mark for each row

Examiner report missing
9618_s21_qp_23 Question 1

2(a)	 One mark for: 1 All four boxes correctly labelled and positioned 2 Selection diamond (only on `UpdateLoan` and no iteration arrows) 3 Parameters to and from `LoanExtend()` 4 Parameters to and from `CheckReserve()` 5 Parameters to and from `LoanReturn()`	**5**

Examiner report missing
9618_s21_qp_23 Question 2

4(a)		**5**

Name	Flag	Index	NewName	ThisChar
"∇in∇a∇∇Cup"				
	TRUE	1	""	
				'∇'
			"∇"	
		2		'i'
	FALSE		"∇I"	
		3		'n'
			"∇In"	
		4		'∇'
	TRUE		"∇In∇"	
		5		'a'
	FALSE		"∇In∇A"	
		6		'∇'
	TRUE		"∇In∇A∇"	
		7		'∇'
			"∇In∇A∇∇"	
		8		'C'
	FALSE		"∇In∇A∇∇C"	
		9		'u'
			"∇In∇A∇∇Cu"	
		10		'p'
			"∇In∇A∇∇Cup"	
	FALSE	11	"∇In∇A∇∇Cup"	'p'

Mark as follows:

- One mark for each of columns 2 to 5 (condone missing final 11)
- One mark for final row all correct (including final 11)

4(c)(i)	A couple of solutions:
	
	`24  ThisChar ← LCASE(MID(Name, Index, 1)`
	
	ALTERNATIVE:
	
	`31  NewName ← NewName & LCASE(ThisChar)`
	
	Ignore line number
		1

4(c)(ii)	One mark for each: Line number: 26 New position: Move to after line 27 / line 28	**2**

Examiner report missing
9618_s21_qp_23 Question 4

<table>
<tr>
<td>2(a)(i)</td>
<td>
<table>
<tr><td>The number of transitions that result in a different state</td><td>3</td></tr>
<tr><td>The number of transitions with associated outputs</td><td>2</td></tr>
<tr><td>The label that should replace ‘X’</td><td>Start</td></tr>
<tr><td>The final or halting state</td><td>S3</td></tr>
</table>
One mark per row
</td>
<td>4</td>
</tr>
</table>

<table>
<tr>
<td>2(a)(ii)</td>
<td>Number of outputs: 1

Current state: S2</td>
<td>2</td>
</tr>
</table>

Examiner report missing
9618_s21_qp_22 Question 2

5(a)(i)	`DECLARE RNum : ARRAY[1:100] OF INTEGER`
	`DECLARE Index, Count : INTEGER`
	
	`Count ← 0`
	`FOR Index ← 1 TO 100`
	`  RNum[Index] ← INT(RAND(200)) + 1`
	`  IF RNum[Index] >= 66 AND RNum[Index] <= 173 THEN`
	`  Count ← Count + 1`
	`  ENDIF`
	`NEXT Index`
	`OUTPUT Count`
	
	Mark as follows:
	1 Array declaration
	2 Loop for 100 iterations
	3 Array element index 'syntax' (left-hand side of assignment expression) **in a loop**
	4 Use of `RAND()` to generate value in range (and assign to array element) **in a loop**
	5 Check if random number within range **and** if so, increment count **in a loop**
	6 Output of count (following a reasonable attempt) **after the loop**
		6

5(a)(ii)	One mark per bullet / sub-bullet point 1 Initialise the array to a rogue value (to indicate 'unassigned' element) 2 Add a conditional loop to: 3 Generate and store a random number (in the correct range) 4 Check the stored number against values already in the array 5 If the stored number is found then generate another random value 6 Otherwise add it to the array (and exit loop) **Note: Max 3 marks**	**3**

Examiner report missing
9618_s21_qp_22 Question 5

7(a)	Test string should contain: • At least one leading space (before the first word) • at least one trailing space (after the last word) • at least one instance of multiple spaces between words • At least one upper case character Mark as follows: One mark for one correct Two marks for three correct Three marks for all correct	**3**

7(b)	One mark for each: • Dry run / produce a trace table / walk through the code • Add output statements to allow the code to be tracked • Insert a Breakpoint into the program // use single-stepping to execute instructions // monitor variables using a watch window • Try different test values to see which ones fail **Note: Max 2 marks**	**2**

Examiner report missing
9618_s21_qp_22 Question 7

1(a)		**4**

Example value	Explanation	Variable name	Data type
`"Wong"`	The preferred name of the member joining the football club	**`MemberName`**	**`STRING`**
`FALSE`	A value to indicate whether an existing member of the club lives at the same address	**`FamilyMember`**	**`BOOLEAN`**
`19/02/1983`	When the member joined the football club	**`StartDate`**	**`DATE`**
`1345`	The number of points a member has earned. Members of the club earn points for different activities.	**`Points`**	**`INTEGER`**

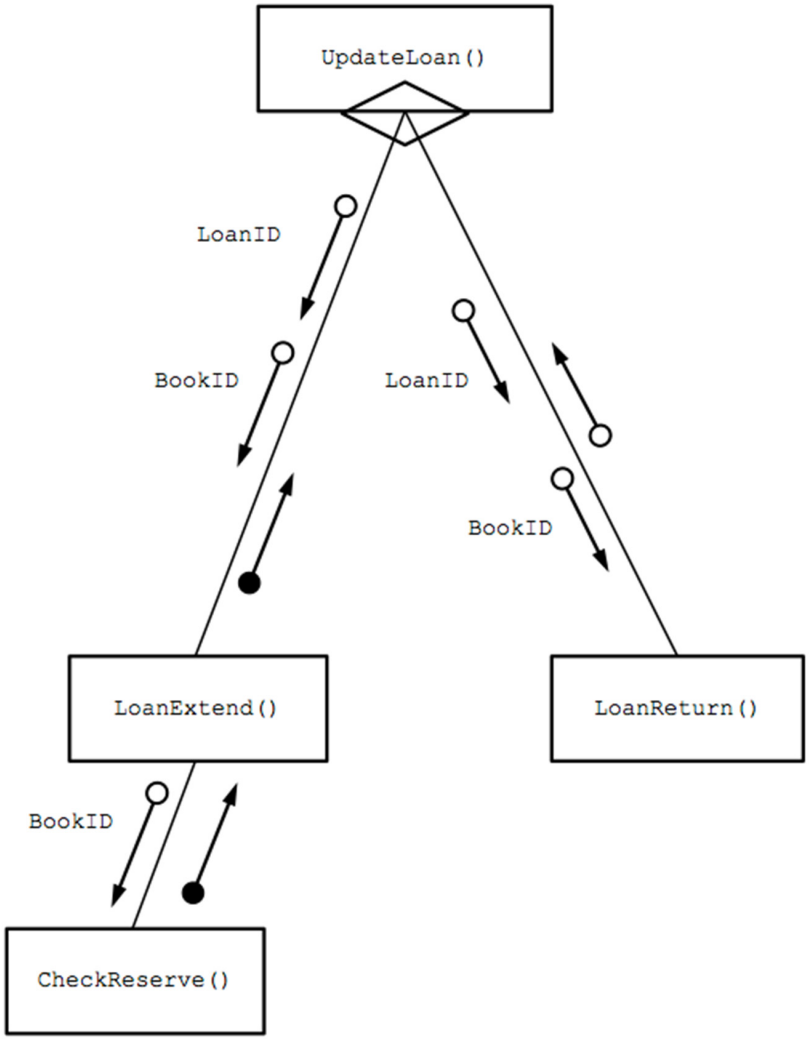
One mark for each appropriate variable name plus data type

1(b)		**5**

Statement	Error
`Result ← 2 & 4`	Should be arithmetic operator (not &) // 2 and 4 should be CHAR / STRING
`SubString ← MID("pseudocode", 4, 1)`	NO ERROR
`IF x = 3 OR 4 THEN`	Not Boolean values / incorrect operator // Condition incorrect
`Result ← Status AND INT(X/2)`	`INT(X/2)` doesn't evaluate to a boolean value / incorrect operator
`Message ← "Done" + LENGTH(MyString)`	Can't add string to number / `"Done"` is not a number

One mark for each row

Examiner report missing
9618_s21_qp_21 Question 1

2(a)	 One mark for: 1 All four boxes correctly labelled and positioned 2 Selection diamond (only on `UpdateLoan` and no iteration arrows) 3 Parameters to and from `LoanExtend()` 4 Parameters to and from `CheckReserve()` 5 Parameters to and from `LoanReturn()`	**5**

Examiner report missing
9618_s21_qp_21 Question 2

4(a)		**5**

Name	Flag	Index	NewName	ThisChar
"∇in∇a∇∇Cup"				
	TRUE	1	""	
				'∇'
			"∇"	
		2		'i'
	FALSE		"∇I"	
		3		'n'
			"∇In"	
		4		'∇'
	TRUE		"∇In∇"	
		5		'a'
	FALSE		"∇In∇A"	
		6		'∇'
	TRUE		"∇In∇A∇"	
		7		'∇'
			"∇In∇A∇∇"	
		8		'C'
	FALSE		"∇In∇A∇∇C"	
		9		'u'
			"∇In∇A∇∇Cu"	
		10		'p'
			"∇In∇A∇∇Cup"	
	FALSE	11	"∇In∇A∇∇Cup"	'p'

Mark as follows:

- One mark for each of columns 2 to 5 (condone missing final 11)
- One mark for final row all correct (including final 11)

4(c)(i)	A couple of solutions:
	
	`24  ThisChar ← LCASE(MID(Name, Index, 1)`
	
	ALTERNATIVE:
	
	`31  NewName ← NewName & LCASE(ThisChar)`
	
	Ignore line number
		1

4(c)(ii)	One mark for each: Line number: 26 New position: Move to after line 27 / line 28	**2**

Examiner report missing
9618_s21_qp_21 Question 4